have at least a loopback interface). The `ifTable` is the *interface table*, which is a list of all network interfaces on the system. The field `ifEntry` shows one particular entry, and `ifDescr` means that we're looking at a description of this interface. This MIB could be expressed as "Interface number 1 on this machine is called `em0`."

MIBs can also be expressed as numbers, and most SNMP clients do their work natively in numerical MIBs. Your management tool should be able to translate between numbers and names, but just so you're not terribly surprised, here's the earlier example in numerical form:

```
.1.3.6.1.2.1.2.2.1.2.1 = STRING: "em0"
```

Expressed in words, this MIB has five parts separated by dots. Expressed in numbers, the MIB has 11 parts. Aren't they supposed to be the same thing? Well, the numerical MIB is longer because it includes the default address .1.3.6.1.2.1, which translates to *.iso.org.dod.internet.mgmt.mib-2*, the standard subset of MIBs used on the Internet. Most SNMP MIBs start with this string, so the management tools no longer bother printing out this name.

MIB Definitions

OpenBSD supports two groups of MIBs:

- The standard host MIBs, which every network management system understands. This information includes network and disk space utilization, software running on the system, and so on.
- MIBs for OpenBSD-specific functions, such as the packet filter, network failover, bridging, and so on. Most network management systems will not understand the OpenBSD-specific MIBs out of the box, so you'll want to teach your management system about OpenBSD's MIBs.

MIBs are defined according to a very strict syntax documented in MIB files. For example, `snmpd` includes MIB files for the OpenBSD-specific functions in */usr/share/snmp/mibs*. These files are written in plaintext, in the very stilted and formal ASN.1 syntax. While you can read and interpret them with nothing more than your brain, I highly recommend copying them to your network management workstation and using an SNMP client to examine them.

MIB browsers interpret MIB files and present them in their full tree-like splendor, complete with definitions of each part of the tree and descriptions of each MIB, taken from the MIB files. Generally speaking, you enter a MIB in the MIB browser, which displays its numerical and word descriptions, and offers the ability to query an SNMP agent for that MIB.

If you don't already have a MIB browser on your OpenBSD workstation, use the `mbrowse` package. If you don't want a graphical interface, use the `net-snmp` package for a full assortment of command-line SNMP client tools, but be prepared to type some long command lines.

SNMP Security

The most common alternate acronym for SNMP is "Security? Not My Problem!" This is unkind, but true. You should use SNMP only behind firewalls or on trusted networks. If you must use SNMP on the naked Internet, employ packet filtering to keep the public from querying your SNMP service. SNMP agents run on UDP port 161, so allow your management hosts access to that port on only your SNMP hosts.

SNMP provides basic security through *communities*. If you read the SNMP documentation, you'll see all kinds of explanations of why a community is not the same as a password, but as far as a sysadmin is concerned, a community is a password.

Most SNMP agents have two communities by default: `public` (read-only access) and `private` (read-write access). OpenBSD's `snmpd` daemon supports both of these communities by default. One of your first tasks will be to change these community names to something that the whole world doesn't know. Just like passwords, community names should be hard for intruders to guess and easy for you to remember.

As you might expect, there have been various versions of SNMP. Version 1 was the first attempt. Version 2c (SNMPv2c) is the more commonly deployed update. Version 3 (SNMPv3) uses encryption to protect data on the wire, and it includes strong authentication. In practice, few vendors actually use it because it's very complicated. The `snmpd` daemon has partial support for SNMPv3. Here, we'll focus on the completely supported SNMPv2c.

Configuring snmpd

Configure `snmpd` in */etc/snmpd.conf*. The configuration format is a series of text statements. Defining new community strings overrides the defaults of `public` and `private`.

We start by defining new read-only and read-write community strings, as follows:

```
read-only community hansteen
read-write community henning
```

In general, most `snmpd` configuration statements look like these two. The `snmpd.conf(5)` man page lists all valid *snmpd.conf* configuration statements.

Every SNMP system is expected to list a contact, a description, and a location, as in this example:

```
system contact "mwlucas@michaelwlucas.com"
system description "Web server"
system location "Rack Row 9, Cabinet 6, Under the Meal Replacement Bars"
```

Many network management systems will automatically pull in this information to populate the database. Here, I've defined these values for my system. Make similar entries for your system.

The default *snmpd.conf* listens to only the localhost IP address, 127.0.0.1, so outside hosts cannot contact the SNMP daemon. If you want to listen on all available addresses, comment out the lines, like the following, that specify an address.

```
listen_addr="127.0.0.1"
listen on $listen_addr
```

Alternatively, you can give an interface IP address to have `snmpd` listen to a specific external IP address for those machines with many addresses.

```
listen_addr="192.0.2.5"
```

With this configuration, `snmpd` can provide information about your system. Enable it in */etc/rc.conf.local*.

```
snmpd_flags=""
```

This will start `snmpd` at boot, or you can run */etc/rc.d/snmpd*.

Debugging snmpd

SNMP can be an annoying protocol to debug. For one, because it's UDP, there's no easy way to test connectivity to the agent. Also, it runs fairly silently, in that it doesn't log queries.

To verify that queries from your network management system are reaching your server, try running `snmpd` in verbose mode and with debugging.

```
# snmpd -vd
startup
snmpe_bind: binding to address 0.0.0.0:161
```

When an SNMP query reaches your server, you should see the server parse the requests. By the same token, `snmpd` is very good about telling you why it can't provide an answer.

```
snmpe_parse: 192.0.2.197: wrong read community
```

Errors, like the following, that arise from requests for a nonexistent MIB are a little more difficult to understand.

```
snmpe_parse: 192.0.2.197: SNMPv1 'henning' context 1 request 1141724535
snmpe_parse: 192.0.2.197: oid iso.org.dod.internet.private.enterprises.2041
snmpe_parse: 192.0.2.197: SNMPv1 'henning' context 0 request 1141724536
snmpe_parse: 192.0.2.197: oid iso.org.dod.internet.private.enterprises.2041
snmpe_parse: 192.0.2.197: invalid varbind element, error index 1
```

Here, the MIB request is trying to find the object identifier (OID) `iso.org.dod.internet.private.enterprises.2041`, but OpenBSD's `snmpd` does not support that. (It does support `2021`, part of the Net-SNMP MIB.) The SNMP client is requesting an invalid MIB.

This example shows a successful request and the MIB that `snmpd` sends in response:

```
snmpe_parse: 192.0.2.197: SNMPv1 'henning' context 1 request 1531862688
snmpe_parse: 192.0.2.197: oid iso.org.dod.internet.private.enterprises.ucDavis
```

By reading the output carefully, you should be able to see why `snmpd` is not answering requests as expected.

Getting snmpd Information

The most important feature of SNMP is that it lets you read statistics from the operating system and/or software. In addition to the usual features supported by SNMP, such as resource utilization and processes, `snmpd` lets you grab OpenBSD-specific system information. You can get information about the packet filter, sensor data, interface memory, and Command Address Redundancy Protocol (CARP). All of this appears under the .1.3.6.1.4.1.30155 MIB, OpenBSD's private (enterprise) MIB tree.

The PF SNMP MIB

The OpenBSD packet filtering feature keeps a lot of statistics, and everything I've ever wanted is available through the PF MIB. You'll find information such as the following:

- Whether PF is on, and how long has it been running (in hundredths of a second)
- The number of packets that have matched filter rules
- The number of fragments and reassembled packets
- The number of packets dropped because of memory problems, internal packet-filtering problems, overfilling the state tables, and so on
- The number of states added and removed from the state table
- The number of timeouts of various protocols
- The amount of traffic blocked on each interface
- Packet filtering table usage, number of addresses in each table

And there's more. The PF SNMP MIB gives you more useful visibility into packet filtering. Point your MIB browser at the .1.3.6.1.4.1.30155.1 MIB to see everything.

Sensors

You can view the same kernel values processed by `sensorsd(8)` (see Chapter 15) via `snmpd`, including a list of sensors on this device, the value reported by the sensor, and whether each sensor is in an alarm state. This means you can use `snmpd` instead of `sensorsd` to monitor your hardware.

To view sensor data via SNMP, examine the MIB tree .1.3.6.1.4.1.30155.2.

Interface Memory

You can view the amount of memory used by an interface, and how often (if ever) an interface was starved for memory as a result of system load. View the MIB tree .1.3.6.1.4.1.30155.5 to see these values.

CARP

CARP is an OpenBSD invention for sharing one address between two or more machines. It was designed to provide highly available IP services. The `snmpd` daemon exposes CARP's innards, including these items:

- The name of each CARP interface
- CARP configuration values (preemption, `advskew`, and so on)
- The number of IPv4 and IPv6 packets received
- The number of packets discarded for various reasons
- The number of times the host has become master

To see the CARP MIB tree, view .1.3.6.1.4.1.30155.6.

Other MIBs

The `snmpd` daemon is constantly being expanded. According to the MIB files, they've reserved space for IPsec and `relayd(8)`. Check */usr/share/snmp/mibs* for additional MIB files, and use your MIB browser to see what your specific version of OpenBSD supports. The OpenBSD team adds MIBs as they're needed and as code is contributed. If you need IPsec MIBs, feel free to write and submit the code.

The SSH Server sshd

Secure Shell (SSH) is a protocol for building encrypted tunnels between hosts. SSH is most commonly used for remote command-line access to a system, but you can use it as a generic wrapper around other protocols or even to build virtual private networks. One common use for SSH is to support secure file transfer protocol service, or SFTP, which doesn't give you a shell prompt but does encrypt files and authentication information as they cross the network.

The OpenBSD project supports OpenSSH, a freely licensed client and server. OpenSSH is the most widely deployed SSH server in the world, with roughly 97 percent market share, and is generally considered the standard SSH server. Entire books have been written about OpenSSH, including mine (*SSH Mastery,* Tilted Windmill Press, 2012).

OpenBSD includes the OpenSSH server `sshd(8)`, the OpenSSH command-line client `ssh(1)`, and the SFTP client `sftp(1)`. We'll focus on `sshd` here, since you can use any number of SSH clients. The ones I use most commonly are `ssh` (for Unix-like systems) and PuTTY (for Windows). For SFTP, I commonly use `sftp` (for Unix-like systems) and WinSCP (for Windows).

Disabling sshd

Unless you specified otherwise during installation, OpenBSD starts `sshd` by default. If you don't want `sshd` to run, disable it in */etc/rc.conf.local*.

```
sshd_flags=NO
```

SSH Host Keys

The first time you start `sshd`, OpenBSD creates *host keys* in */etc/ssh*. These are sets of public and private keys that uniquely identify an SSH server. Each key file includes the word *key* in its name. When your client first connects to the SSH server, it presents a fingerprint summary of the server's host key. If you tell the client to accept the key, the client will cache the server's host key. If this key ever changes, the client warns the user that the server's unique identity has changed, and that the user might be offering his login credentials to a different server. (Anyone who gets copies of the host keys can have another server masquerade as yours.) Be sure to back up your host keys, and protect them from theft.

sshd Network Options

You could change `sshd`'s behavior by adding command-line flags, but the most common way to reconfigure `sshd` is to edit the files in */etc/ssh*.

OpenSSH has many configuration options. The ones that are most commonly changed involve the network settings. You can control the port, IP address, and version of IP `sshd` listens to by editing the configuration file */etc/ssh/sshd_config*. Here's an example:

```
Port 22
AddressFamily any
ListenAddress 0.0.0.0
ListenAddress ::
```

The `Port` keyword specifies the TCP/IP port that `sshd` attaches to. The default is TCP port 22.

NOTE *Some people recommend using a port other than 22 to avoid password-guessing worms. Far better ways to protect your SSH server are to allow only public-key authentication or use a packet filter to allow logins from only selected hosts or networks.*

The `AddressFamily` keyword specifies the version of IP that `sshd` uses. The default is to use both IPv4 and IPv6, but you can restrict it to a specific protocol with the `inet` (IPv4) or `inet6` (IPv6) keyword.

Lastly, you can attach `sshd` to a specific IP address with the `ListenAddress` option.

chrooting Users

Organizations commonly need to confine users to a particular directory or subset of directories. For example, many websites allow users command-line access over SSH so that they can edit their files and debug problems more easily, or even just SFTP access to their files. Those users should have access to their own directories, but not to other users' files, or any other part of the system. One solution is to `chroot` the user in his home directory. If you have several users who need to access a shared directory, you can `chroot` all of them in that directory.

Locking users in a directory involves three steps: choosing the directory to lock users into, populating that directory, and configuring `sshd` to `chroot` those users. To demonstrate, we'll walk through an example of `chroot`ing the user `lasnyder` in his home directory, and give him command-line access, so he will be able to access only the programs in his `chroot`.

Choosing the Directory

First, specify the `chroot` directory with the `ChrootDirectory` option.

```
ChrootDirectory /home/lasnyder
```

This works well if all of your users need to be locked into the same directory, but if you want users to have their own private directory, or if you want to specify a directory elsewhere on the filesystem, things get more complex.

OpenSSH supports the `%%`, `%h`, and `%u` macros to represent home directories. If your `chroot` directory includes a literal `%`, use the `%%` macro to represent it. The server in this example has home directories on */disk%3/home*, so the `%%` macro is needed to escape the percent sign.

```
ChrootDirectory /disk%%3/home/lasnyder
```

The %u macro expands to the user's username. You could use this to give users a chroot some place other than their home directory (though I don't know why you wouldn't just give them a home directory in the desired location). Here, each user has a directory under */var/www*:

```
ChrootDirectory /var/www/%u
```

Finally, you could lock each user in his home directory with the %h macro.

```
ChrootDirectory %h
```

Wherever you lock a user, you must give that directory everything the user needs to function, since the user won't be able to leave that directory to get a tool that he might need.

Populating the chroot

Most programs, such as a shell, require at least a few device nodes, and the user must have a shell program to be able to run one. If a user has only SFTP access, you don't need to do any special preparation of the chroot. OpenSSH's SFTP server includes everything it needs. But if users have shell access, they need basic device nodes and a shell program.

For our example, to give lasnyder what he needs, go to the chroot directory, create a *dev* directory, and then make the standard device nodes using /dev/MAKEDEV. You can remove the console, klog, kmem, ksyms, mem, and xf86 devices.

```
# cd /home/lasnyder
# /dev/MAKEDEV std
# rm console klog kmem ksyms mem xf86
```

Now we need to get the user a shell. Since programs running inside the shell cannot access any files outside the chroot, including shared libraries, any shell copied into a chroot must be statically linked. The included system shells are statically linked, and most shells in the ports tree can be built in static flavors.

Verify that a shell is statically linked with file(1), and then create a *bin* directory inside the chroot and copy the shell there.

```
# file /bin/ksh
/bin/ksh: ELF 32-bit LSB executable, Intel 80386, version 1, for OpenBSD, statically linked, stripped
# cd /home/lasnyder
# mkdir bin
# cd bin
# cp /bin/ksh .
```

Lastly, although a chrooted user should not have write access to his own root directory, he needs a real home directory. The user's home directory in */etc/passwd* is relative to the chroot; in other words, if a user's home directory

in */etc/passwd* is */home/lasnyder*, and the user is chrooted to */home/lasnyder*, his personal files and dotfiles actually go in */home/lasnyder/home/lasnyder*.

```
# chown root:wheel /home/lasnyder
# mkdir -p /home/lasnyder/home/lasnyder
# chown lasnyder:lasnyder /home/lasnyder/home/lasnyder
```

The user now has a command-line friendly jail cell on the system. Now we need to tell sshd to lock the user in it.

chrooting Specific Users

Applying this chroot strategy to all of your users probably isn't advisable—if nothing else, your sysadmins need unfettered system access to perform maintenance.

To tell sshd to chroot specific users, either by name or by group, use the Match keyword at the end of *sshd_config*. Match lets you change sshd's default behavior based on factors such as user and client IP address. (Match has many more functions; see sshd_config(5) for examples.)

For example, if you wanted to chroot only the user lasnyder, you could use Match to specify his username. Early in the configuration, you would have a ChrootDirectory statement that turns off chroot for most users. Then, at the end of the configuration, you would change the setting based on matching that username.

```
...
ChrootDirectory none
...
Match User lasnyder
      ChrootDirectory %h
```

You could also chroot all users in a group.

```
...
ChrootDirectory none
...
Match Group webcustomers
      ChrootDirectory %h
```

If you have multiple Match terms, separate them with commas.

```
...
ChrootDirectory none
...
Match User lasnyder, jgballard, pkdick
      ChrootDirectory %h
```

Or, if most of your users are chrooted, reverse the default and specifically dechroot your sysadmins.

```
...
ChrootDirectory %h
...
Match Group wheel
      ChrootDirectory none
```

With careful configuration, you can restrict access to only the desired users.

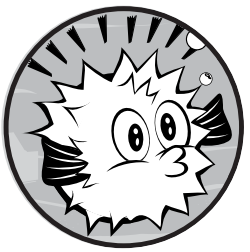
SSH can do a whole lot more, such as securely eliminate passwords from your network. It's worth your time to fully master this protocol.

OpenBSD's built-in services can help you hold your network together, and they provide all kinds of useful support infrastructure. Now that you know how to configure some of these built-in programs, let's see how to use OpenBSD as a desktop.

17

DESKTOP OPENBSD

Spend summer days with
blowfish at your fingertips:
no passwords stolen!



OpenBSD is best known as a server operating system, but it can be a very effective and powerful desktop system. The X Window System is the standard graphic desktop software for Unix-like operating systems, and OpenBSD includes tools for using it. As this book assumes that you have some Unix experience, I won't cover all the applications that make X Windows comfortable. You'll need to experiment to find your preferred mail client, web browser, and text editors—most of which aren't OpenBSD-specific. Instead, this chapter covers items that are unique to OpenBSD, originated with OpenBSD, or require specific configuration.

OpenBSD includes the Xenocara framework for modifying and building X.Org in a manner tightly integrated with OpenBSD. We'll discuss making OpenBSD boot into a graphical desktop using the `cwm` desktop environment, as well as a text console using the `tmux` terminal multiplexer. But we'll start by customizing the console.

Configuring Your Console with wscons

The `wscons(4)` hardware-independent console driver lets you configure your boring, black-and-white, nongraphical console in many ways.

Start by viewing the current console settings using `wsconsctl(8)`. Run the following on a text console, not in an X session (changes made in `wscons` can carry over to an X session, but once you start X, you're mostly stuck with X's configuration system).

```
$ wsconsctl
keyboard.type=pc-xt
keyboard.bell.pitch=400
keyboard.bell.period=100
keyboard.bell.volume=50
...
```

Each line contains a system variable and a setting, many of which you can change. The `keyboard.type` variable represents the type of keyboard on the system. Because this is an amd64 system, it uses the `pc-xt` keyboard common to consumer computers, but you'll see different keyboard types with different hardware.

You can also change these settings with `wsconsctl`. For example, in the previous listing, the variable `keyboard.bell.volume` sets the volume of the computer's beep. Now, I'm a `tcsh` user, and I frequently use tab completion (type a character or two, press TAB, and the shell fills in the name of the command or file you're about to type). Unfortunately, when tab completion hits an ambiguous spot, it stops. That's not a problem when I'm logged in over SSH, because I can just type a character or two and press TAB again. But when I'm on the local console, each ambiguity is accompanied by a beep (or bell) from the computer. When I'm trying to fix a problem, and the bell rings, shouting "*beep* WRONG! *beep* WRONG! *beep* WRONG!" I have only one thought:

The beep must die.

```
$ wsconsctl keyboard.bell.volume=0
keyboard.bell.volume=0
```

Now silence reigns and I can resolve the problem without the computer nagging me. (You could choose to turn up the volume, if you're a masochist. I won't judge you—at least not in public.)

Here, we'll look at a couple other things you can do with `wscons`.

Screen Blanking

If you leave the system alone for a few minutes, the screen should go blank to reduce power usage. Modern monitors often do this on their own, but

you can also configure this behavior in the operating system, especially for older platforms. OpenBSD turns off the display only once it knows how to reactivate the display.

You have three choices:

`display.kbdact` Wake on keyboard activity

`display.msact` Wake on mouse activity

`display.outact` Wake on monitor output

Set one of these `wscons` variables to `on`, and OpenBSD will realize it should start blanking the monitor after the idle timeout. The variable `display.screen_off` gives the idle timeout in milliseconds; the default, `600000`, is 10 minutes.

You can also choose between turning the screen black and putting the monitor into "power-saver" mode, also known as *sleep*. A monitor showing a black screen reactivates immediately when triggered, but uses more power. A sleeping monitor really is off, and needs a few seconds to reactivate. To set power-saver mode, change the variable `display.vblank` to `on`. (Some old monitors don't believe in saving power, so this won't work on them.)

Setting wscons Variables at Boot

Users can adjust console settings, but those settings will disappear at the next reboot. To set `wscons` variables at boot, add them to */etc/wsconsctl.conf*. The boot process reads this file and applies any variables it finds to the console.

Running Virtual Terminals with tmux

The terminal multiplexer `tmux(1)` lets you run multiple virtual terminals inside one OpenBSD terminal window. While standard virtual terminals disappear when you disconnect from the system, `tmux` virtual terminals continue to run even after you disconnect. `tmux` is small, fast, easy to use, and written with the same care as the rest of OpenBSD.

Why would you need `tmux`? One example is for building programs. Just before I leave the office, I use my laptop to make an SSH connection into an OpenBSD server, create a virtual terminal, start building a huge program (such as OpenOffice.org), and shut down my laptop. Normally, the build on the server would terminate when my session is interrupted, but the `tmux` virtual terminal continues to run even when I log out. The build continues in the disconnected virtual terminal while I drive home, and when I reconnect to it later, I can see how the build has progressed. Virtual terminal sessions even survive accidental disconnections caused by network or client failures.

This section provides an introduction to `tmux`. For complete details on the features discussed here, as well as dozens of other features, read `tmux(1)`.

The tmux Status Bar and Window Names

To start a virtual terminal session, run `tmux`. Your terminal window will show the command prompt and a green `tmux` status bar along the bottom, with information like the following:

```
[0] 0:ksh*        "caddis.blackhelicop" 11:55  26-Jun-13
```

This is a virtual terminal session. The left side of the status bar displays the `tmux` session number in brackets `[0]` and the list of `tmux` windows `0:ksh*` (beginning with window number 0). The right side shows the first part of your machine name (`caddis.blackhelicopters.org`), followed by the time and date. You'll learn how to customize things in "Setting tmux Options" and "Configuring tmux" on page 329.

The window name defaults to the name of the program running in that `tmux(1)` window. For example, if you start a command that continues until interrupted, such as `iostat -w 5`, the session name will change to match the command. Interrupt the command, return to a shell prompt, and the status bar should change its name to match your shell.

The status bar is normally green, but if it turns yellow, `tmux` is expecting input. When it's yellow, any typing is interpreted as a `tmux` command. If you reach this mode accidentally, press ENTER to return to a green status bar and normal operation.

tmux Commands and Window Management

Pressing CTRL-B tells `tmux` that the next command is for `tmux`, not for the program running in the virtual terminal. (If pressing CTRL-B interferes with another program you use frequently, you can change this key combination, as you'll see in "Unmapping and Remapping Keys" on page 336.)

The most commonly used `tmux` commands are single characters. For example, to create a second terminal window in this `tmux` session, press CTRL-B-C. Your screen will display only a command prompt and a new status bar.

```
[0] 0:iostat-  1:ksh*          "caddis.blackhelicopte" 11:58 26-Jun-13
```

You have two windows: window 0 shows `iostat` output, and window 1 displays the `ksh` prompt. The asterisk next to window 1 means that you're currently looking at it. Run an ongoing command in your new window, such as `top`, and the window name should automatically change to the name of that command.

Changing the Current Window

To view another window, use one of the following key combinations:

- To see the next window, press CTRL-B-N.
- To switch to the previous window, press CTRL-B-P.

NOTE *Keep in mind that window ordering wraps. For example, if you are on the last window and press* CTRL-*B-N, you should see the first window.*

- To jump directly to a window by number, press CTRL-B followed by the window number.
- To open a menu of all windows, press CTRL-B-W, and then select a window with the arrow keys.

I find the next and previous sequences sufficient, but if you end up with a dozen windows in one terminal, you might think otherwise.

Renaming Windows

Terminal windows take the name of the currently running program, but that's not always useful. For example, if I'm compiling the newest source with `make build`, the window name will continually change to reflect the command running in the build at that moment. The only problem is that the constant flickering change in my status bar drives me nuts.

If you don't want to see the window name change with each command, use CTRL-B to assign a static name to the window. A yellow `[rename-window]` prompt will appear in the status bar. Enter your preferred window name, such as `upgrade`, and then press ENTER.

Terminating Windows

To kill a window and end any processes running in it, change to that window and press CTRL-B-&. You will get a confirmation prompt.

Getting Online Help

Press CTRL-B-? to see a complete list of all `tmux` commands.

```
   C-b: send-prefix
   C-o: rotate-window
   C-z: suspend-client
  Space: next-layout
    !: break-pane
    ": split-window
    #: list-buffers
...
```

Now you can easily explore `tmux` without reading the manual page. You'll use this list to remap keys in "Unmapping and Remapping Keys" on page 336.

Disconnecting, Reconnecting, and Managing Sessions

A collection of `tmux` windows is called a *session*. Conveniently, `tmux` can disconnect from a running session without interrupting its windows. Press

CTRL-B-D to disconnect your terminal from the current tmux session. Your terminal should now show what it held before starting tmux. To reconnect to your tmux session, run tmux attach.

You can have multiple tmux sessions simultaneously. The session number appears on the far left of the status bar. (In our sample status bars, the tmux session is 0.)

To start a new tmux session without attaching to your previous session, run tmux without any arguments. For example, I type tmux instead of tmux attach in order to spawn a new tmux session when I want to pick up where I left off. You can change your tmux session within tmux itself, using a tmux command, but I usually just end the session and enter the correct command.

If you can have all these tmux sessions, how can you be sure that you haven't left old, useless sessions lying around, with abandoned commands running in them? Use tmux list-sessions.

```
$ tmux list-sessions
0: 4 windows (created Sun Feb 13 12:17:14 2011) [80x23]
2: 1 windows (created Mon Feb 21 21:57:59 2011) [131x36] (attached)
```

I can see from the last line of this output that I left session 2 running on my other workstation, and am still attached to it.

To connect to session 2, use attach-session and option -t to choose a target session. Here, I attach to tmux session 2:

```
$ tmux attach-session -t 2
```

I'm now connected to the same session from two separate SSH sessions—in this case, from two separate client workstations. My typing in one screen is echoed on the other.

To destroy a session, use the kill-session command, specifying the session number with -t. Here, I kill tmux session 2:

```
$ tmux kill-session -t 2
```

Any programs running in windows in tmux session 2 will also be killed.

Using tmux Commands

Command mode in tmux offers a prompt for entering more complicated commands. To enter command mode, press CTRL-B-:. The status bar will turn yellow, and a single colon replaces all window names. For example, to create a new window dedicated to running systat(1), press CTRL-B-: and enter **neww systat**. A window named systat will appear. Switch to that window, and then press CTRL-C to stop systat. That window will disappear.

You can do all sorts of things with tmux commands, including split windows into multiple panels, copy and paste text, and so on. (Read tmux(1) for the full list.) If you want to cut and paste from one window to another,

it's easiest if you use multiple terminal windows, but if you are working in a text-only console or another restricted environment, you might find these tmux features useful.

The tmux command mode is most commonly used to set options.

Setting tmux Options

Options change how tmux windows, sessions, and the tmux server itself behave. The most common changes involve the appearance of windows, colors, or items displayed in the status bar. Some options affect the entire tmux session; others affect only a specific window. You can change options on the fly with the tmux command set-option.

Go ahead and open a tmux session to follow along. Press CTRL-B-: to enter command mode. When the colon appears, enter **set-option status-fg green**, and then press ENTER. Your status bar should now be solid green bar. Congratulations! You've set the status bar text color identical to the background color, making it unreadable. Return to command mode, and change the color to black to make it readable again. (If this bugs you, you can kill this tmux session and start a new one to reset all options.)

When making changes, use **set-option** (or just set) for options that affect the tmux server and the entire session. Use **set-window-option** (abbreviated setw) for options that affect only a single window.

Most people won't need many (if any) tmux options, but they can prove useful. For example, say you want the status bar clock to display time in 24-hour format, or you want a visual bell instead of a beep. Options let you control these behaviors, as well as run commands in the status bar. To change basic tmux appearance and behavior, see the options in tmux(1).

Be sure to try any interesting options interactively. Once you have a tmux session running the way you like, enter **show-options** for an accurate list of the current options. Copy that list because we'll use it to build a configuration file.

Configuring tmux

Modify *$HOME/.tmux.conf* in your home directory to configure your tmux sessions, or use */etc/tmux.conf* to inflict your tmux preferences on every system user. Personal tmux configurations override global settings.

As a simple example, I've set the left side of my status bar (containing the session number) to blue, and the right side (the hostname, time, and date) to red. If I decide I like this, I can make this change permanent by entering the following in *tmux.conf*:

```
set -g status-left-bg blue
set -g status-right-bg red
```

The -g flag sets an option globally, so it takes effect for all sessions and windows.

This should get you comfortable with using tmux. If you need multiple terminal windows simultaneously, use a graphical desktop. Stay tuned.

Setting Up X

The OpenBSD developers modified the industry-standard X graphic interface provided by X.Org to better fit with OpenBSD. The combination of X.Org and OpenBSD-specific patches is called Xenocara.

In most cases, Xenocara works exactly like X.Org, and X.Org documentation is applicable to OpenBSD. Most of Xenocara is there for security and for the convenience of developers building X, but there are a few additions. In my opinion, OpenBSD's best enhancement to X.Org is the `cwm(1)` window manager. Here, we'll cover configuring and starting X. The next section provides details about using the window manager.

Configuring X

Configuring X can be simple or agonizing, depending on your hardware.

Most video cards require special access to system memory, though some new Intel video cards can work without this access. For other cards, you must adjust the `machdep.allowaperture=2` sysctl in */etc/sysctl.conf* and reboot.

Most amd64 and i386 systems need `machdep.allowaperture` set to `2`, but other platforms might require `1` or `2`. Without this sysctl setting, the kernel will not permit X to communicate with the graphics card. If you're in doubt, try X without changing the sysctl, and when you find out your hardware is too old or the wrong model to work that way, set it to `2`.

After rebooting, see if X.Org can automatically set up your graphics interface by running `startx`. If it works, you should see the `fvwm(1)` desktop and a very bland gray background with a couple of terminal windows.

NOTE *If X doesn't start, see the OpenBSD FAQ, the X.Org documentation, and* /var/log/Xorg.0.log. *Many things can go wrong with X autoconfiguration. Read your error log and search the Internet for solutions.*

Once you know that X works, it's time to decide whether you want to start X manually each time you need it or if you want OpenBSD to boot in to X automatically.

Starting X Manually

After logging into the text console, run `startx(1)`. This command starts the commands in *$HOME/.xinitrc* and starts X.

Booting into X

OpenBSD includes an */etc/rc.conf* hook for starting the X Display Manager, `xdm(1)`, at boot time with a login prompt, but the default is to not use `xdm`. Here's the line to add to your *rc.conf.local* to have `xdm` without any flags start X at boot:

```
xdm_flags=""
```

After booting, the console will show a graphical login prompt. Once a user logs in, xdm runs any commands in *$HOME/.xsession*.

NOTE *Because this is a chapter about using OpenBSD as a desktop, I assume that you're using xdm(1). Examples refer to* $HOME/.xsession. *If you use startx(1) instead, substitute* .xinitrc.

Emulating a Three-Button Mouse

A lot of X software expects you to have a mouse with three or more buttons, but many have only two buttons. Xenocara lets you pretend that you have a third mouse button, and when you press both mouse buttons simultaneously, it interprets that as pressing the nonexistent third button.

Of course, the best solution is to buy a real mouse with three or more buttons. They're much easier to get than they used to be.

Now that you have X ready to use, let's explore that cwm window manager I mentioned earlier.

Using the cwm Window Manager

While X provides operating system support for a graphical interface, management of that interface falls to the window manager. OpenBSD has packages for button-heavy, pointy-clicky window managers such as KDE, Gnome, and Xfce. These window managers might provide a comfortable bridge between consumer-friendly operating systems and OpenBSD, but they're not designed for the more hard-core Unix user.

Xenocara includes three window managers: the classic fvwm(1) and twm(1) window managers that have shipped with X since the last millennium, and the OpenBSD-specific cwm(1). OpenBSD developers wrote cwm specifically as a modern, fast, keyboard-friendly interface.

To start cwm at login, invoke it in *$HOME/.xsession*:

```
/usr/X11R6/bin/cwm
```

When your cwm session ends, xdm returns you to the login screen.

Configuring cwm

Rather than using mouse-driven configuration menus, cwm uses a single configuration file, *$HOME/.cwmrc*. You can read the complete documentation in cwmrc(5). Here, as I discuss various cwm features, I'll mention how each can be configured or changed in *.cwmrc*.

Modifier Keys

Most cwm operations require you to press a configurable combination of keys. For example, CTRL-ALT-DEL locks the screen. The cwm documentation lists the modifier keys shown in Table 17-1.

Table 17-1: cwm Modifier Keys

Symbol	Key
C	CTRL
S	SHIFT
M	META/ALT

For example, `CS-r` in *.cwmrc* means CTRL-SHIFT-R. `CM-W` represents CTRL-ALT-W.

Choosing a New Window Manager

The default `cwm` configuration allows you to choose a new window manager with CTRL-ALT-W from any command on the system. Enter **`cwm`**, and `cwm` should restart without losing any of your windows.

NOTE *You can also enter a command that isn't a window manager, such as `grep`. If you do, OpenBSD will silently log you out. It won't say, "Please step away from the keyboard before I hurt you." Not threatening you passes for user-friendly in OpenBSD.*

Binding a Key Sequence to a Command

You can also bind a key sequence to any `cwm` command listed in `cwmrc(5)`. For example, suppose you want to use the key sequence CTRL-ALT-R to delete your current window. Add the following to *.cwmrc*:

```
bind CM-r                delete
```

The change will take effect only once you use CTRL-ALT-W to restart `cwm` or you log out and back in again. After you've done one or the other, use CTRL-ALT-R to delete the current window.

WARNING *If you make an entry in* .cwmrc *that `cwm` cannot parse, `cwm` will not process the configuration file, and you will lose all of your custom `cwm` settings as soon as you load the configuration file. If your custom settings vanish, your most recent changes to* .cwmrc *are wrong. If you make an error that* `cwm` can *parse, `cwm` will accept it. No one except the user will have trouble in this case.*

Creating cwm Windows

When you are running the graphical desktop, everything on screen is a *window*. A terminal runs in a window, as do web browsers and games. Managing windows—raising, hiding, resizing, naming, and so on—is the core task of a window manager.

A default `cwm` session starts with a plain gray screen and a small `xconsole(1)` window. Create a new terminal window with CTRL-SHIFT-ENTER. The window

manager should focus on whatever window your mouse is over. (Press SHIFT-+ to increase the font size of the terminal windows.)

If you press CTRL-SHIFT-ENTER repeatedly, you won't see additional terminal windows. Oh, the new windows will be created, but on top of one another. Press ALT and the left mouse button to move the currently active window, and you should expose another terminal window beneath that one.

I find the default terminal size too small; I want wider terminals with more rows. To resize the terminal window, press ALT and the center mouse button (or both buttons simultaneously). The mouse will move to the lower-right corner of the window and change to a right angle bracket. The window will continue to resize as long as you hold down the mouse button.

To maximize windows vertically, press CTRL-ALT-=. To maximize windows horizontally, press CTRL-ALT- SHIFT-=. To destroy a window, focus on it and press CTRL-ALT-X. You will not be asked to confirm your decision; `cwm` will obey and exterminate the window immediately.

To exit `cwm` and return to the login screen, press CTRL-ALT-Q.

Managing Windows

Now that you can create windows, let's look at ways to manage them.

First, switch between visible windows with ALT-TAB. The newly active window should rise to the foreground.

To assign a name to a window, press CTRL-ALT-N to access the label prompt, and then enter the window's desired name. Names are useful when you choose to hide a window without destroying it.

To hide a window, focus on it, and then press ALT-ENTER to make it disappear. Pressing ALT-TAB won't bring it back because it's hidden. Press the left mouse button for a list of all hidden windows, arranged by name. Any windows you didn't name will show up as the program name. All terminals show up as `xterm`. Click the name to unhide the window.

NOTE *It's a good idea to name windows that have a specific purpose, such as a long-running software build. That way, you can minimize the window when it's not interesting, and quickly find it again when necessary. I name windows created by SSH sessions after the connected server.*

You can search for windows by name. Press CTRL-ALT-/ to get a `window>>` prompt, and then start typing the name of the window. `cwm` will list all matching windows. Hidden windows have an ampersand (`&`) before their name. Exclamation points indicate the window with focus.

Locking the Screen

Don't walk away from an active workstation without locking it, especially if you're logged on to sensitive systems or as root. Press CTRL-ALT-DEL to lock your desktop, and the screen will go blank. Press another key, and `cwm` will request your password to unlock the workstation.

The default screensaver is a blank screen, provided with `xlock(1)`. To use a different screensaver, set a path to it in *.cwmrc*, as follows:

```
command lock path-to-command
```

For example, to use `xlock`'s flow mode as a screensaver, add the following to *.cwmrc*:

```
command lock '/usr/X11R6/bin/xlock -mode flow'
```

If you don't like any of the screensavers in Xenocara's `xlock(1)`, try the `xscreensaver` package.

NOTE *`xlock` is easily bypassed by anyone with console access. You can't count on it for security, but it does make a decent reminder for your coworkers.*

Connecting to Other Machines with SSH

One common task is to connect to remote machines with SSH. To do so, press ALT-. to display an `ssh>>` prompt, and then enter the name of the machine to which you want to make the connection. Conveniently, `cwm` supports autocompletion, based on entries in *known_hosts*. As you type a hostname at the `ssh` prompt, `cwm` checks for matching names in the system's and users' *known_hosts*. Press the down-arrow key to find your desired hosts, or keep typing the hostname to connect to a new host. (Autocompletion won't work if you hash *known_hosts* entries.)

And by the way, if you open multiple SSH sessions, name them, because sorting through multiple sessions labeled `ssh` is annoying.

Creating an Application Menu

Click the right mouse button on the background to bring up the application menu. The `cwm` developers have no idea which programs are important to you, so they don't even try to provide a default application menu. You need to build that yourself with *.cwmrc* entries. Each command has the following format:

```
command name path-to-command
```

Yes, this is exactly the same as the format for setting the screensaver. The `lock` command is actually one of two special command keywords. Here, I've created an application menu with two choices, which are my web browser and my PDF reader:

```
command firefox /usr/local/bin/firefox
command xpdf /usr/local/bin/xpdf
```

When I right-click the desktop background, I'll see a menu with these two choices.

Using Keyboard Navigation

Almost everyone has a mouse these days, but sometimes you're in a situation where it's best to ignore that mouse. Perhaps your desk is too small, your mouse is broken, repeatedly removing your hands from the keyboard slows you down (as in you don't have a sensible mouse-in-keyboard), or you just hate your mouse today (which is both valid and respectable).

To control the mouse cursor with the keyboard, use CTRL and the arrow keys to move the pointer a small amount, or press CTRL-SHIFT and an arrow key to make larger pointer movements. On my system CTRL-SHIFT-up arrow moves the mouse pointer up about a terminal line, but that varies with font size.

You can also use keyboard commands to shift window placement and size, as shown in Table 17-2.

Table 17-2: cwm Window Movement Direction Keys

Key Combination	Direction
ALT-SHIFT–H	Left
ALT-SHIFT–J	Down
ALT-SHIFT–K	Up
ALT-SHIFT–I	Right

Use ALT and a direction key to move a window a small amount, or ALT-SHIFT to move the window a larger amount. To resize a window by a small amount, use CTRL-ALT and a direction key. CTRL-ALT-SHIFT and a direction key resizes the window a larger amount. Just as if you were resizing with the mouse, the size change occurs from the lower-right corner of the window. Place a window's upper-left corner where you want it, and then resize the window.

Decorating cwm

The default `cwm` desktop is rather bland, but a few adjustments make it easier on the eyes. One of the first things I set is a background color: black. Use `xsetroot(1)` to set your background color.

```
$ xsetroot -solid black
```

You can include this command in *.xsession* or run it in a terminal. The file */usr/X11R6/share/X11/rgb.txt* lists the colors X recognizes. If a color name is two words, either remove the spaces in the name or put the name in single quotes, like this:

```
$ xsetroot -solid 'hot pink'
```

If you want an image in the background, use `feh` (*/usr/ports/graphics/feh*).

```
$ feh --bg-scale /home/mwlucas/galaxies.jpg
```

To make window edges easier to identify, put borders around them. The default border is 1 pixel wide, in your choice of colors. I prefer 3-pixel borders, blue for the active window and dark blue for the inactive windows. That requires the following entries in *.cwmrc*:

```
borderwidth 3
color activeborder blue
color inactiveborder darkblue
```

As you grow more accustomed to `cwm`, you might find that you want particular applications—perhaps an MP3 player, a clock, and a fancy graphic system load indicator—to always be visible. Maximizing a window can bury these applications. To address this, define a gap in *.cwmrc*, which sets the number of pixels to be kept clear even when you maximize a window.

```
gap top bottom left right
```

For example, when I must keep track of time, I run `xclock(1)` on the right side of my screen. Experimentation has shown that my clock is about 175 pixels wide. I leave a gap of 180 pixels, so that even when I maximize a window, it doesn't cover the clock. Here's my `gap` entry in *.cwmrc*:

```
gap 0 0 0 180
```

Now I can no longer use the excuse that I missed work because I lost my clock on my desktop. Fortunately, I have many other handy excuses.

Unmapping and Remapping Keys

While the `cwm` authors did their best to choose keyboard shortcuts that wouldn't conflict with those used by other programs, they could not avoid every possible conflict. If you run into such a conflict, you can solve the problem by modifying entries in *.cwmrc* to replace conflicting `cwm` key bindings.

For example, `cwm` uses CTRL and CTRL-SHIFT with the arrow keys to move the pointer, but OpenOffice also uses these key combinations to move the pointer and highlight within a text document. I've used OpenOffice for more than 10 years, and have written millions of words in it. My fingers have been well-trained, and I'm not going to try to retrain them. The `cwm` key assignments must change.

Use the `bind` command to remap keys. Start by disconnecting the CTRL and CTRL-SHIFT and arrow key combinations from `cwm` with the `unmap` option. Remember that *.cwmrc* uses `C` to represent CTRL and `S` to represent SHIFT (as shown earlier in Table 17-1).

```
bind CS-Left unmap
bind CS-Right unmap
bind CS-Up unmap
bind CS-Down unmap
bind C-Left unmap
bind C-Right unmap
bind C-Up unmap
bind C-Down unmap
```

These keystrokes will now pass through to applications, such as OpenOffice.

To determine how to move the pointer with the keyboard, I check `cwmrc(5)` for the list of commands that can be bound to a key. The manual defines commands with a brief name and a description of their functionality. The pointer movement commands begin with `ptrmove` and `bigptrmove`, plus a direction. I find them and use the Windows key (also known as modifier 4) to replace the functions I removed from the CTRL key.

```
bind 4-Left ptrmoveleft
bind 4-Right ptrmoveright
bind 4-Up ptrmoveup
bind 4-Down ptrmovedown
bind 4S-Left bigptrmoveleft
bind 4S-Right bigptrmoveright
bind 4S-Up bigptrmoveup
bind 4S-Down bigptrmovedown
```

I can now use both OpenOffice and `cwm`'s keyboard functions.

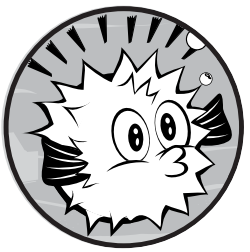
At this point, I've covered everything I've used since OpenBSD introduced `cwm`, which should get you started. For more information, read `cwm(1)` and `cwmrc(5)`. You'll see that `cwm` supports many more features.

Now that we've covered OpenBSD's appearance, let's dive deep into the operating system core.

18

KERNEL CONFIGURATION

Kernel, not colonel!
This is blowfish, not chicken.
Less grease, more function.



Depending on your systems administration experience and background, the kernel is a subject of great mystery and speculation. It might be something you reconfigure when the whim strikes you, or something you know to leave alone.

Most commercial operating systems provide only a few basic hooks for configuring the kernel. Many open source operating systems tell you to rebuild the kernel from source code whenever you change anything.

OpenBSD falls somewhere in the middle.

The standard OpenBSD kernel is intended to be perfectly usable without modifications, but you have the tools to perform any tweaks or adjustments necessary for your environment. Additionally, you have the complete source code and kernel-building tools in case you decide to perform wholesale kernel surgery.

OpenBSD lets you adjust kernel behavior even as the system is running, via `sysctl(8)`. Some hardware or protocols require special OpenBSD kernel tweaks to run in specific environments. This chapter will cover both kinds of changes, but first, let's talk about the kernel in general.

What Is the Kernel?

"The file */bsd* is OpenBSD's kernel. Next question?"

That's technically correct, but not exactly useful. A more general description is that "The kernel is the interface that links applications and the hardware." That's not a complete definition, but it's good enough.

The kernel allows programs to write data to disk drives and to the network, and it gives instructions to the CPU and shuffles bits into memory. When you open a web page, the browser application asks the kernel to fetch the data it displays.

Some kernel responsibilities exceed this definition. For example, the kernel handles network connectivity, including forwarding packets from one interface to another if needed. The packet-filtering rules run in the kernel (although the rules are managed by applications). The kernel handles disk redundancy. And the kernel also handles all sorts of things that don't impact applications but are integral to a functioning system.

A simplified view is to think of the kernel as the program that handles all the low-level functions, which is close enough to give you an idea of what the kernel does.

Along with *kernel*, you'll also hear the term *userland*. Userland is everything in the system that isn't the kernel. Your shells, libraries, and applications are all part of userland.

Kernel Messages

The kernel issues messages to userland. These include hardware attaching and detaching alerts, warnings from device drivers, and system boot messages. If you're logged on to the system console in text mode, you might notice these messages.

To review kernel messages, you can watch the console, check the system logs (as discussed in Chapter 15), or use `dmesg(8)`.

OpenBSD has a system message buffer, where it sends messages from the kernel. These messages are usually copied to the system logger, but they're also accessible via `dmesg`.

The system message buffer is circular. As it fills up, the oldest messages are deleted to make room for new ones. Run `dmesg` to view it.

Startup Messages

One common question is "What hardware did your kernel find?" If the kernel handles all the device drivers and other hardware support, the list of devices found should include all the supported hardware in the system.

While the system message buffer is circular, OpenBSD copies the boot-time system messages into */var/run/dmesg.boot*. Here are the boot messages from one of my test systems.

```
OpenBSD 5.2-current (GENERIC) #287: Tue Aug 21 18:15:00 MDT 2013
    deraadt@i386.openbsd.org:/usr/src/sys/arch/i386/compile/GENERIC
cpu0: AMD Opteron(tm) Processor 4184 ("AuthenticAMD" 686-class, 512KB L2 cache)
2.80 GHz
cpu0:FPU,V86,DE,PSE,TSC,MSR,PAE,MCE,CX8,APIC,SEP,MTRR,PGE,MCA,CMOV,PAT,
PSE36,CFLUSH,MMX,FXSR,SSE,SSE2,NXE,MMXX,FFXSR,LONG,3DNOW2,3DNOW,SSE3,CX16,
POPCNT,LAHF,ABM,SSE4A
real mem  = 267907072 (255MB)
avail mem = 252616704 (240MB)
...
```

The first line lists the version of OpenBSD, the kernel name and version, the date the kernel was built, as well as the machine and directory where the kernel was built and who built it. This machine runs an official OpenBSD i386 snapshot, built by Theo de Raadt.

We then see some specifics on the processor. Those familiar with AMD will note that this is a 64-bit amd64 processor. I chose to run the 32-bit i386 version of OpenBSD, because that's the installation disk I had handy.

This system came with 256MB of RAM, but 1MB is lost due to hardware-level weirdness. OpenBSD sees 255MB, and 240MB are available to programs other than the kernel at this moment. The kernel might use some of that memory later.

Device Attachments

The kernel then explores the hardware. When it finds hardware that matches a device driver, it attaches the device driver to the hardware.

```
mainbus0 at root
bios0 at mainbus0: AT/286+ BIOS, date 10/13/09, BIOS32 rev. 0 @ 0xfd780,
SMBIOS rev. 2.4 @ 0xe0010 (98 entries)
bios0: vendor Phoenix Technologies LTD version "6.00" date 10/13/2009
bios0: VMware, Inc. VMware Virtual Platform
acpi0 at bios0: rev 2
```

OpenBSD found the main system bus, `mainbus0`, which is a bit odd because it's not actually a piece of hardware. The kernel creates this logical device as a point for all other devices to attach to. It's not the only logical device driver, but it's present on every machine.

The `bios0` device, for the hardware BIOS, isn't terribly interesting either. You know the hardware has some kind of BIOS. We covered configuring your system BIOS back in Chapter 3, and you haven't needed to look at it since. Similarly, the `acpi0` device represents the Advanced Configuration and Power Interface (ACPI). If it needed any configuration, you took care of that after unpacking the system from the shipping box.

Connections and Numbering

Now we get into real hardware.

```
pci0 at mainbus0 bus 0: configuration mode 1 (bios)
pchb0 at pci0 dev 0 function 0 "Intel 82443BX AGP" rev 0x01
ppb0 at pci0 dev 1 function 0 "Intel 82443BX AGP" rev 0x01
pci1 at ppb0 bus 1
piixpcib0 at pci0 dev 7 function 0 "Intel 82371AB PIIX4 ISA" rev 0x08
pciide0 at pci0 dev 7 function 1 "Intel 82371AB IDE" rev 0x01: DMA, channel 0
configured to compatibility, channel 1 configured to compatibility
```

The first PCI bus, device `pci0`, is attached to `mainbus0` in the slot bus 0. The kernel then finds a device it identifies as `pchb0`, and attaches it to the PCI bus as device 0. Don't know what `pchb0` is? Use `man pchb` to identify this as a PCI host bridge. `dmesg` gives you the part number.

Next is the device `ppb0` (a PCI/PCI bridge, per `ppb(4)`), attached to PCI bus 0 as device 1. This is followed by another PCI bus, `pci1`, attached to the `ppb` device. Each instance of a device is assigned a number, starting with zero. Our tenth PCI bus would be device `pci9`. (There's no technical requirement for sequential numbering, but the kernel follows this rule unless you tell it otherwise.)

If you dig through *dmesg.boot*, you'll see that every device is plugged into another device somewhere. For example, here's my keyboard.

```
wskbd0 at pckbd0: console keyboard, using wsdisplay0
```

The keyboard `wskbd0` is attached to device `pckbd0`.

```
pckbc0 at isa0 port 0x60/5
pckbd0 at pckbc0 (kbd slot)
```

Device `pckbd0` is attached to device `pckbc0`, which, in turn, is plugged into the `isa0` device, which is the ISA bus.

```
isa0 at piixpcib0
```

The ISA bus is connected to the Intel PIIX4 ISA bridge.

```
piixpcib0 at pci0 dev 7 function 0 "Intel 82371AB PIIX4 ISA" rev 0x08
```

And this bridge is then hooked to PCI bus 0.

OpenBSD finds devices from the root outward, which means that everything is listed in the reverse order from what you've just seen. You get a list of which devices are attached to a device, and then the devices attached to those devices. You can backtrack starting with the end device, but that's kind of annoying.

Using dmassage to View Installed Devices

I find the `dmassage` package most useful for identifying exactly what's attached to what devices, although that's not its only function. Install `dmassage` like any other package, and then run it with `-t` to display installed devices as a tree, like this:

```
root
 |-mainbus0
 |  |-bios0
 |  |-cpu0
 |  |-ioapic0
 |  |-pci0
 |  |  |-mpi0
 |  |  |  \-scsibus1
 |  |  |     \-sd0
 |  |  |-pchb0
 |  |  |-pciide0
 |  |  |  \-atapiscsi0
 |  |  |     \-scsibus0
 |  |  |        \-cd0
...
```

While this information may not be immediately useful, `dmassage` illustrates how devices are interconnected on your system, which may become important later.

Viewing and Adjusting Sysctls

As noted in earlier chapters, the OpenBSD kernel includes a variety of parameters known as *system controls*, or *sysctls*. Some sysctls are static and can be viewed but not changed. The root account can change others, either at runtime or at boot.

Sysctls allow an application to retrieve information from the kernel. They also let a sysadmin change system behavior without reconfiguring applications, recompiling the kernel, or rebooting. You can view sysctl values and adjust those that can be changed with `sysctl(8)`.

That said, just because you *can* change sysctls doesn't mean you *should* change them. The OpenBSD developers set the sysctls to default values that work well for most environments. You might need to change one or two for your system, but if you find yourself changing sysctls all over the place, you're probably sending yourself down the sysadmin rabbit hole.

Sysctl MIBs

The kernel presents sysctls in a MIB tree. As you learned in Chapter 16, MIB trees organize information into hierarchical categories. The top-level categories include `kern` (kernel), `vm` (virtual memory), `net` (networking), `hw` (hardware), `machdep` (machine-dependent values), and so on. Each of these categories has additional subcategories. For example, `net` has the categories

inet (IPv4) and inet6 (IPv6). The inet6 MIBs have subcategories ip6 (general IPv6 characteristics) and icmp6 (ICMP for IPv6). When you reach the end of categories, you'll find individual MIBs like these:

```
net.inet6.ip6.forwarding=0
```

This MIB configures forwarding IPv6 packets between interfaces, turning the host into the router. How do I know? I've read it in the documentation, and it's a commented example in */etc/sysctl.conf*. OpenBSD doesn't maintain a central list of sysctl values, but the man pages refer to any related sysctls.

If you want to explore sysctls, get a list from your system, as described next.

Viewing Sysctls

Use sysctl(8) to view the sysctls available on a system.

```
$ sysctl
kern.ostype=OpenBSD
kern.osrelease=5.2
kern.osrevision=201211
kern.version=OpenBSD 5.2-current (GENERIC) #287: Tue Aug 21 18:15:00 MDT 2013
    deraadt@i386.openbsd.org:/usr/src/sys/arch/i386/compile/GENERIC
...
```

This particular system has more than 400 sysctls. Interpreting the kern.ostype and kern.osrelease sysctls is fairly straightforward, but why would an OpenBSD system have a sysctl to report the operating system?

The sysctl(3) interface appears in all BSD-derived operating systems and even in Linux, so checking the kern.ostype sysctl, or checking for its existence, is a good way for third-party software to identify the operating system. kern.osrevision is just the year and month this particular snapshot was built. kern.version is the kernel compilation information displayed at boot. That's not hard, is it? Let's look at the next few sysctls:

```
kern.maxvnodes=5926
kern.maxproc=1310
kern.maxfiles=7030
kern.argmax=262144
```

Figuring out what these do is a little harder than interpreting the previous sysctl names. An experienced sysadmin could make really good guesses about these, but guessing isn't system administration. Always research sysctls before changing them.

When you know the name of a sysctl and you want to view its current value, give the sysctl name as an argument to sysctl. For example, to view the current securelevel (discussed in Chapter 10), check the kern.securelevel sysctl.

```
$ sysctl kern.securelevel
kern.securelevel=1
```

The current value of `kern.securelevel` is 1.

You can view subsets of the sysctl tree by giving just the part of the tree you're interested in. For example, to view only the sysctls related to ICMP, check the `sysctl net.inet.icmp` subcategory.

```
$ sysctl net.inet.icmp
net.inet.icmp.maskrepl=0
net.inet.icmp.bmcastecho=0
net.inet.icmp.errppslimit=100
net.inet.icmp.rediraccept=0
net.inet.icmp.redirtimeout=600
net.inet.icmp.tstamprepl=1
```

OpenBSD has six sysctls for IPv4 ICMP networking. You can view any portion of the sysctl tree this way, going as deep or as shallow as you like.

Changing Sysctl Values

Some sysctls are read-only. For example, the `hw.ncpufound` sysctl shows how many processors the system has.

```
$ sysctl hw.ncpufound
hw.ncpufound=1
```

This system has one processor. You cannot change the number of hardware processors through software (duh).

On the other hand, a system decides whether or not to forward packets in software. OpenBSD performs packet forwarding entirely in the kernel, like embedded firewalls and routers. The sysctl `net.inet.ip.forwarding` controls this feature. If this is set to `0`, packets are not forwarded. If it's set to 1, the system routes packets.

```
$ sysctl net.inet.ip.forwarding
net.inet.ip.forwarding=0
```

To change this, use the equal sign to assign a new value.

```
# sysctl net.inet.ip.forwarding=1
net.inet.ip.forwarding: 0 -> 1
```

If you need to stop forwarding packets, set this sysctl to `0`.

Changes take effect immediately. Remember that only root can change sysctl values.

Types of Sysctl Values

Most sysctls have a numerical value, but the interpretation of that number depends on the sysctl. A few sysctls are words, and some generate tables.

Numerical Sysctls

Some sysctls are Boolean—either on or off. For example, IP forwarding is either on or off. You can't have 50 percent packet forwarding on a properly functioning system.

Other numerical sysctls have a range of valid numbers. For example the `kern.securelevel` sysctl can range from `-1` to `2`, as discussed in Chapter 10. While you could assign a value outside this range, it wouldn't have any effect beyond the closest valid value.

Some sysctls have numerical values that map directly to some kernel value. For example, the `kern.maxproc` sysctl gives the maximum number of processes that the system can run. You can adjust this value as needed to support your applications. While there's no maximum value, increasing `kern.maxproc` increases the memory used by various in-kernel tables. By the same token, there's no minimum size, but if you reduce this setting too far, the system won't run correctly.

Word Sysctls

A few sysctls are words, such as the `kern.ostype` sysctl examined earlier. Most of these sysctls cannot be changed with `sysctl`, but some can be changed with other programs. For example, the sysctl `kern.hostname` gives the system's hostname. You cannot change `kern.hostname` with `sysctl`, but you can change it with `hostname(8)`.

Table Sysctls

In addition to words and numbers, some sysctls generate output in the form of tables. These sysctls are not intended for direct human consumption, but are meant for processing by dedicated userland programs. For example, `netstat(1)` reads table sysctls to create its output.

To view all sysctls, including tables, pass the `-A` option to `sysctl`.

```
$ sysctl -A
```

Many table sysctls still won't print (they will generate warnings that you should use program such-and-such to view that data), but you'll get a few tables amid the regular output.

And by the way, tabular sysctls are read-only.

Setting Sysctls at Boot

Sysctl changes are not permanent; they revert when you reboot. To make sysctl changes permanent, set them in */etc/sysctl.conf*.

Changes specified in *sysctl.conf* take place early in the booting process, before any server software starts. For example, if you need to customize the network stack, those changes should take place before the system opens any network connections. List the sysctls you need to change, an equal sign, and the desired value in *sysctl.conf*.

The default *sysctl.conf* contains commonly changed sysctls (those that the OpenBSD team expects you might reasonably want to change). Each is commented out with a pound sign (#) and set to the most common nondefault setting. If you want to change the sysctl, uncomment the entry.

The following are some commonly changed entries from *sysctl.conf*. (You might have different entries in your system, depending on your OpenBSD version.)

net.inet.ip.forwarding
: This controls forwarding of IPv4 packets between interfaces. When set to 1, the system forwards packets received on any interface according to the internal routing table. When set to 0 (the default), packets are not forwarded.

net.inet.icmp.rediraccept
: This determines whether the host will accept ICMP redirects. Routers send ICMP redirects to direct hosts to use different local gateways for more specific routes. While the router can forward the packets for the clients, using redirects reduces network load. Accepting ICMP redirects means the host could be redirected to an invalid gateway, however, so they can be a security issue. Set this to 1 to accept ICMP redirects. The default of 0 ignores ICMP redirects.

net.inet6.ip6.forwarding
: This controls the forwarding of IPv6 packets, much like net.inet.ip.forwarding does for IPv4 packets. You can control IPv4 and IPv6 forwarding separately. Set this to 1 to forward IPv6 packets.

net.inet6.icmp6.rediraccept
: By default, OpenBSD ICMPv6 ignores redirects, just as it ignores IPv4 ICMP redirects. Set this to 1 to accept ICMPv6 redirects.

net.inet6.ip6.accept_rtadv
: IPv6 autoconfiguration listens for router advertisements, much as IPv4 autoconfiguration listens for configurations from DHCP servers. To autoconfigure IPv6, a host must accept router advertisements. Set this to 0 to disable accepting router advertisements.

net.inet.tcp.always_keepalive
: The TCP keep-alive feature sends packets over otherwise idle connections so that intermediate devices will recognize that a connection is still in use. Proper firewalls recognize live but idle TCP connections even without keep-alives. If you have a broken firewall or NAT device, TCP keep-alives can help hold a connection alive. Set this to 1 to enable keep-alives.

net.inet.tcp.ecn
: By default, OpenBSD's TCP stack does not use Explicit Congestion Notification (ECN). Set this to 1 to enable ECN.

ddb.panic
: OpenBSD uses the ddb(4) kernel debugger. If you want the system to drop into the debugger in the unlikely event of a kernel panic, leave this at 1. If you want the system to reboot as soon as possible, set this to 0.

ddb.console
: When set to 1, this enables entering the ddb(4) debugger from the console when someone presses CTRL-ALT-ESC. This option is primarily of interest to developers.

vm.swapencrypt.enable
: By default, OpenBSD encrypts all data written to swap. To disable encrypting swap, set this to 0. There's really no reason to disable swap encryption, because encrypting swap space induces minimal system load.

machdep.allowaperture
: This controls userland program access to the memory that userland really shouldn't be able to access. The X Windows System needs access to this memory to display a graphical console. (Chapter 17 covers this sysctl and X.)

machdep.kbdreset
: On amd64 and i386 systems, setting this to 1 allows you to press CTRL-ALT-DEL on the console to do a clean shutdown and reboot. When set to 0 (the default), pressing CTRL-ALT-DEL has no effect.

As a rule, if you don't understand the thing that a sysctl affects, don't change it. You won't learn about RFC 3390 by playing with a sysctl related to it; you'll learn about RFC 3390 by actually *reading* RFC 3390 and spending quality time with a packet sniffer watching traffic with RFC 3390 disabled and enabled.

And if you want to change a sysctl that's not listed here, think twice. If the OpenBSD guys wanted you to change it, they would list it in *sysctl.conf*.

Altering the Kernel with config(8)

While sysctl lets you tweak the kernel, it won't let you change values that are hard-coded into the kernel binary. Some of these values are used to initialize kernel data structures, and they can't be changed once the kernel is running. Others relate to device drivers. Once the kernel has finished probing devices, it won't go back and reprobe just because you change where a device driver checks for its hardware. To change hard-coded values like these, you must edit the existing kernel file and reboot, allowing the system to set things as you like from initialization. That's where config(8) comes in.

The config command has two completely separate functions. The first creates a kernel compilation directory from a text configuration file, as

discussed in Chapter 19. The function we're most interested in now is editing an existing kernel binary, which lets you tweak a kernel to better suit your needs.

NOTE *The modern OpenBSD kernel is largely dynamic. If you call for additional virtual interfaces, the kernel creates them. If you need to change the amount of memory for the buffer cache, use a sysctl. Editing the kernel is rarely necessary.*

Making a Backup of the Default Kernel

Before making any changes to a working kernel, no matter how minor, back up the original kernel! If your minor changes make your machine unbootable, you want to be able to easily fall back to a working kernel.

The kernel is just a file, */bsd*. To back it up, copy it to another file. I recommend naming your backup of the default kernel */bsd.GENERIC*, for reasons that will become apparent in Chapter 19.

Always keep a known-good kernel on your system. A bad kernel can prevent a computer from booting, and if you don't have a reliable kernel that's easily bootable, you will need to boot from installation media. (Boot your backup kernel using the instructions in Chapter 5.) And remember that subtle kernel bugs can take weeks or months to show up, so plan to keep your backup kernel forever.

Device Drivers and the Kernel

Much of the hard-coded information in the kernel relates to device drivers, especially drivers for ancient ISA cards.

Some of you may remember manually configuring the interrupt request (IRQ) and memory port addresses on a network or SCSI card. The kernel uses the IRQ to identify cards. Essentially, it consults an internal list of IRQs and port numbers, compares it to what it finds on the hardware probe, and assigns the drivers appropriately. "This card answers at IRQ 10 and memory port 0x300? It must be a NE2000-compatible network card. I will assign that driver to it." The process is more complicated than this, of course, but this probe is a vital part of the process. If you want OpenBSD to recognize such a card, and the card is set to an IRQ and memory port other than what OpenBSD expects, you must tell the kernel the IRQ and memory port the card is using.

Realistically, the best way to deal with ISA cards is to feed them to the recycling plant. Running OpenBSD on a 25-year-old VAX is interesting and educational. Running OpenBSD on 15-year-old Sparc hardware is realistic for very specific applications, and can also be educational and interesting. Running OpenBSD on 10-year-old consumer-grade i386 hardware is either a waste of time or an exercise in masochism—probably both.

NOTE *Modern PCI-descended hardware includes hooks for the kernel to identify the hardware and assign the proper device driver. You shouldn't need to edit the kernel to support hardware.*

Enabling Drivers

Rather than changing driver IRQs, more realistically, you might need to enable a device driver that's disabled by default or disable a device that's on by default.

The kernel includes some device drivers that are disabled because they react badly with certain hardware, such as the IPMI driver. The `ipmi(4)` driver is known to be buggy, and as I write this, it is badly broken in some use cases. It's included in the default kernel, but disabled by default.

You can choose to enable `ipmi(4)`. If it works for you, great. If it doesn't, feel free to submit bug reports, preferably with patches, or at least proper `dmesg` output and crash dumps.

Editing the Kernel with config

When using `config` as a kernel editor, use the command-line options `-e` and `-o`. The `-e` flag tells `config` you're editing a kernel binary. The `-o` flag lets you specify a new file for the edited version of the kernel.

Give the original kernel file path as an argument. For example, here's how to edit */bsd* and write the result to the file */bsd.test*:

```
# config -e -o /bsd.test /bsd
```

You could use the `-f` flag instead of `-o` and a filename. The `-f` flag tells `config` to edit the kernel file in place, not to create a new file.

NOTE *If you're editing /*bsd *and you specified the -f option, your changes are written directly to /*bsd. *I recommend* not *doing this. (Unless, of course, you're absolutely certain you know what you're doing. You get to keep all the parts.)*

Running `config` will open the kernel editor, which should look much like this:

```
OpenBSD 5.2-current (GENERIC) #287: Tue Aug 21 18:15:00 MDT 2013
    deraadt@i386.openbsd.org:/usr/src/sys/arch/i386/compile/GENERIC
Enter 'help' for information
ukc>
```

At this point, you need to use kernel editor commands to make changes.

Using the help and list Commands

Start with the two editor commands `help` and `list`. The `help` command shows all the commands available within `config` and comes in particularly handy at stupid-o'clock AM to remind you of the necessary syntax.

The `list` command displays a complete list of all the devices the kernel supports, one screen at a time.

```
ukc> list
  0 video* at uvideo* flags 0x0
  1 audio* at uaudio*|sb0|sb*|gus0|gus*|pas0|ess*|wss0|wss*|ym*|eap*|envy*|
eso*|sv*|neo*|cmpci*|clcs*|clct*|auacer*|auglx*|auich*|auixp*|autri*|auvia*|
azalia*|fms*|maestro*|esa*|yds*|emu* flags 0x0
  2 midi* at umidi*|sb0|sb*|ym*|mpu*|mpu*|autri*|eap*|envy* flags 0x0
...
```

On an OpenBSD 5.2 system, the default kernel has 538 entries, most for hardware that isn't on any particular system but that OpenBSD supports out of the box. Let's take a closer look at the devices shown.

Line 0 says that this kernel supports the `video` device. The kernel will look for a video device attached to the `uvideo` device. The `uvideo(4)` man page tells us that `uvideo` is USB video, mainly for webcams and the like, and `video(4)` says that the `video` driver is a device-independent video driver. The `flags` statement gives settings to feed to this device driver. (This kernel supports webcams.)

Line 1 says that this kernel supports an `audio` device, and it can be attached to any of a long list of device drivers. The online manual says that `uaudio`, `sb0`, `gus0`, and so on are sound cards. We get sound with our video? Truly we live in an age of wonders.

Entries for older ISA gear are more complex.

```
278 ne0 at isa0 port 0x240 size 0 iomem -1 iosiz 0 irq 9 drq -1 drq2 -1 flags 0x0
```

This entry for supporting the old-fashioned NE2000 ISA network card includes an IRQ, DRQ, memory port, and a few other settings that I've (thankfully) forgotten about. The kernel will check ISA bus number 0 at the stated port and IRQ, in the hope of finding such a device.

```
504 pflog count 1 (pseudo device)
```

This is a *pseudo-device*—a software creation that acts much like an actual device but has no underlying hardware. The `pflog(4)` pseudo-device is where the packet filter dumps its logs. This kernel creates one instance of the `pflog` device at boot, but thanks to OpenBSD's cloneable interfaces, the kernel can create more `pflog` interfaces as needed.

Finally, notice that several lines declare themselves "free." You can copy an existing device and add it to the kernel. For example, if you wanted a kernel that supported 10 NE2000 cards, and needed 10 instances of the device driver in the kernel, you could copy and add the devices here. The kernel will autoconfigure any number of device driver instances for modern hardware; it will find 10 PCI Express network cards and give them their own instances of the device without any prodding from you.

Finding and Enabling Devices

One of the disadvantages to the `list` command is that it shows everything in the kernel. You can't interrupt it; you must scroll through to the end. It's also difficult to search through several hundred devices by eye. If you know the device you want, use `find` to search for it. Here, we'll use `ipmi` as an example.

```
ukc> find ipmi
493 ipmi0 at mainbus0 disable bus -1 flags 0x0
```

The IPMI device is device number 493, and it is attached to the device `mainbus0`. But note the word `disable` in the device entry. The `ipmi` device is disabled. Let's turn it on.

```
ukc> enable ipmi
493 ipmi0 enabled
```

The kernel now has an active IPMI driver. Yippee!

Changing Kernel Constants

In addition to the device drivers, the kernel has a few hard-coded values for internal data structures. If you run `help` in the kernel editor, you'll see these values as options.

```
ukc> help
...
        bufcachepercent [number]            Show/change BUFCACHEPERCENT
        nkmempg         [number]            Show/change NKMEMPAGES
```

As you can see there are only two values: `BUFCACHEPERCENT` and `NKMEMPAGES`. Unless you have a compelling reason to touch these values, leave them alone.

`NKMEMPAGES` is the number of pages of memory dedicated to the kernel. If your machine starts panicking with error messages of `out of space in kmem_map`, you can increase this value. If the system boots successfully, however, you're better off setting the `vm.nkmempages` sysctl rather than editing the kernel.

`BUFCACHEPERCENT` is the percentage of physical memory dedicated to the buffer cache. In some fairly rare circumstances, increasing the size of the buffer cache can improve filesystem performance. You could set the sysctl `kern.bufcachepercent` instead of editing this kernel value, however.

To view a current value, enter its name.

```
ukc> bufcachepercent
bufcachepercent = 20
```

To change the value, enter its name and the desired value.

```
ukc> bufcachepercent 50
bufcachepercent = 50
```

Again, don't muck with these numbers arbitrarily. The OpenBSD developers set them to the default values for very good reasons.

Completing Configuration

Once you've made all of your changes, enter **quit** to save your changes and write them to a kernel file. The `exit` command discards all changes and leaves the editor, making it easy to start over. Do not mix `quit` and `exit` unless you like being annoyed and confused.

Installing Your Edited Kernel

Your edited kernel is just a file. Verify that you have a backup of your working kernel, copy your new kernel to */bsd*, and reboot.

Boot-Time Kernel Configuration

The `config` kernel editor is great when you know what you're doing, but many of us aren't that lucky or educated. When I'm trying to figure out how to fix a problem, I'll frequently make a change, reboot to test the change, and see if things work.

OpenBSD lets you edit the kernel at boot time. You can try one boot with a kernel change, see if it works, and write your changes to the kernel. At the boot loader prompt, run **boot -c**.

```
boot > boot -c
```

You'll get a couple lines of boot output, and then the kernel editor prompt.

```
ukc>
```

This works just like the `config` kernel configuration editor. Make any changes you want here, exactly as you would with `config`. When you `quit` the editor, the kernel should boot with the changes you've chosen.

The nice thing about boot-time edits is that they're not permanent unless you later declare them so. If your changes don't result in the desired behavior, reboot and try again. If your changes do solve your issue, however, you can write them to a kernel file.

The kernel remembers the changes you made in it. You can "replay" those changes in `config` by using the `-u` flag. Run `config` as if you were editing the kernel, but add the `-u` flag to replicate your boot-time changes.

```
# config -u -e -o /bsd.test /bsd
```

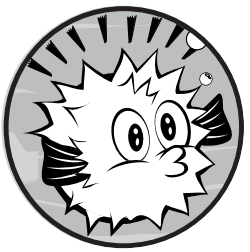
When you get your command prompt, enter **quit** to save your changes to your new kernel file.

Between `sysctl` and `config`, you should be able to make any OpenBSD-supported changes to the kernel. In the next chapter, we'll cover how to make wildly unsupported kernel changes by rebuilding the kernel from source.

19

BUILDING CUSTOM KERNELS

Rewiring the brain?
Knowing where the parts plug in
makes it possible.



The OpenBSD team works very hard to provide a high-quality kernel that requires no tweaking beyond setting the occasional sysctl or perhaps enabling a feature. But if you want to use an experimental feature or add a device driver to the kernel, or you want to squeeze OpenBSD into tiny hardware or embedded systems, you'll need to build a custom kernel from source code. The OpenBSD people won't support you if you venture into custom kernels, but they'll provide you with everything you need to shoot yourself in the foot, as you'll learn in this chapter.

Kernel Cautions

Before we get into the details of building custom kernels, we'll look at why that's usually a bad idea.

Don't Build Custom Kernels

Many open source operating systems encourage sysadmins to build custom kernels. Mailing lists for these operating systems are full of suggestions on rebuilding, tweaking, and modifying the kernel. Those user communities will walk new users through rebuilding the kernel.

OpenBSD developers take a different approach to rebuilding the kernel. They ship a default kernel, called GENERIC, which you will almost never need to rebuild.

Building a kernel from source doesn't prove that you're an alpha geek, and rebuilding the kernel is never a recommended way to solve a problem. The people who build custom kernels are either kernel developers or ignorant newbies. The OpenBSD Project members feel no particular obligation to help users with customized kernels. If your custom kernel crashes, destroys your filesystem, or starts making threatening calls to the local constabulary, they won't care. Why? Adding, moving, or changing one kernel option might seem trivial, but each option might represent tens of thousands of lines of source code that you've just casually gutted.

That said, the OpenBSD Project is much friendlier than closed source operating systems, in that it provides the source code for the kernel, and gives you the tools and instructions needed to build it. The territory might be dangerous, with rattlesnakes and bears and the occasional bottomless pit, but they give you a map and a flashlight. If you can carve out some new territory for yourself, good for you! If you get eaten by coyotes, well, that's pretty much what happens.

NOTE *These warnings apply only to custom kernels. The OpenBSD team is extremely interested in problems in a provided kernel, whether that's the GENERIC kernel, the installer kernel, or any other.*

When working with kernels, keep in mind that some platforms have multiple GENERIC kernels. For example, the i386 platform has the standard GENERIC kernel, but it provides GENERIC.MP for multiprocessor machines, and it supports both versions. By the same token, the SGI platform has several GENERIC kernels—one for each supported hardware variety. These kernels are all GENERIC, and all supported.

Why Build Custom Kernels?

People build custom kernels for various reasons. For example, if you're a kernel developer, or aspire to be one, you will need to build customized kernels to test new features and new code.

Some people who play with kernels are interested in using experimental features. For example, OpenBSD supports the newly developed but not well-tested multipath SCSI, which is not supported by GENERIC. Not many people have the hardware to use multipath SCSI, but those who do have the hardware, along with programming skills, are encouraged to help improve

this feature. (When running experimental features, be sure that you understand that *experimental* is the Siamese twin of *unstable*.)

Rarely, remediating a security flaw will require a patch to the kernel source code. But rather than build your own, get the patch from OpenBSD's stable branch or a snapshot (discussed in Chapter 20).

Finally, some people will build custom kernels to save RAM on a machine with very low memory. Removing features from the kernel reduces its size.

Problems Building Custom Kernels

When building a custom kernel, you are likely to run into trouble. For one, the interdependencies between kernel modules are quite complex and not thoroughly documented. The developers generally assume that people building custom kernels will read kernel source code and man pages. You are expected to read error messages and sort them out yourself.

OpenBSD's cross-platform design slightly complicates kernel configuration. Some devices run on some architectures, but they fail to run or behave weirdly on others. If you include the wrong device in your kernel or tell the kernel a card is attached to the wrong bus, you'll be building a busted kernel. Be sure that you understand how your hardware actually fits together.

When mucking around in the source tree, you can corrupt the source code in various ways, such as by applying a patch incorrectly, scrambling a file, or forgetting that you edited a file that is now causing you grief. To test your source code, compile GENERIC. If GENERIC won't compile, you've either mucked up the source code or your system has some deeper problem.

Building a custom kernel usually means including or removing kernel options and features from the configuration file. If you're trying to use fancy compiler flags, however, stop. Custom compiler options are great for exposing compiler bugs, but the OpenBSD team members make no effort to have their code comply with the demands from these compiler options. Many of these options and higher optimizations break if you're not running very specific operating systems on very specific architectures. The kernel code assumes that you are using the specified compiler options; if you change them, you'll get nothing but pain.

If you've checked everything, and you still can't get your kernel to build, you might don your flameproof suit and ask for help on *misc@OpenBSD.org*. State up front that you're trying to build a custom kernel, and include the following information:

- Your kernel configuration
- OpenBSD version
- Unedited boot-time messages from booting a GENERIC kernel on your computer
- A full description of the problem

Someone might take pity and try to help you.

Problems Running Custom Kernels

Custom kernels can have any number of problems, such as the following:

- Programs might not run as expected.
- The system might not boot.
- The system might crash randomly.
- The kernel might not find all of your hardware.
- The kernel might eat your hard drives or your motherboard (without mustard or even a shot of malt vinegar).

If you have customized your kernel narrowly—say, by adding only the multipath SCSI driver to the GENERIC kernel—the developers working on that feature will probably be interested in your bug reports on that feature.

If you can reproduce that problem when the same system boots with the GENERIC kernel, the OpenBSD team is definitely interested. Report your problem as occurring on the GENERIC kernel, and include debugging output only from GENERIC, not from your custom kernel. If you manage to identify, debug, and create a patch for a problem with a custom kernel, send your patch and a problem description to the mailing list. Your problem may be due to running on a custom kernel, but you may also have found a bug that could be triggered in GENERIC.

But most important, if you have a problem running a custom kernel, reboot with GENERIC and get on with your day.

Preparing for Kernel Customization

Before customizing the kernel, back up the known-good GENERIC kernel on your system by copying */bsd* to */bsd.GENERIC*. That way, if your custom kernel doesn't boot, you can recover by booting the backup kernel.

You'll need the kernel source code in order to build a custom kernel. You can just grab *sys.tar.gz* from your OpenBSD installation media. If you installed from an Internet mirror, make sure to get the source code for your version of OpenBSD. The OpenBSD mirror root directory usually contains a snapshot of fairly recent source code, but check the directory for your release for its source code. Expand this directory under */usr/src*.

```
# cd /usr/src
# tar -xzvpf sys.tar.gz
```

Now that you have a backup (you *did* make a backup of your working kernel when I told you to, right?) and the source code, let's look at kernel configuration.

Kernel Configuration

You configure the OpenBSD kernel via text files. Like 4.4BSD, OpenBSD doesn't offer a fancy graphical kernel configuration utility or menu-driven system. Each kernel configuration is on a single line, along with a label indicating the type of entry and a description. Pound signs (#) mark comments.

Configuration Entries

Kernel configuration entries fall into four general categories: options, device drivers, pseudo-devices, and keywords.

Options

Options are hardware-independent kernel functions. Options handle things like filesystems, networking protocols, and compatibility layers.

Option entries look like this:

```
option     FFS         # UFS
option     INET        # IP + ICMP + TCP + UDP
option     CRYPTO      # Cryptographic framework
```

To learn more about options, read the `options(4)` man page.

Device Drivers

Device drivers give the kernel the necessary software to interact with a piece of hardware. If you want your kernel to support a piece of hardware, it must include the appropriate device driver.

Device driver kernel configuration entries can be quite long. They might include flags or settings that tell the kernel where to find the device and how to initialize it. (ISA cards usually have a hard-coded IRQ and/or memory address.)

Device drivers have no common label, but their entry starts with the device name.

```
mainbus0 at root
cpu0     at mainbus?
fxp*     at pci?                          # EtherExpress 10/100B ethernet
wd*      at wdc? flags 0x0000
ec0      at isa? port 0x250 iomem 0xd8000 irq 9  # 3C503 ethernet
```

Pseudo-Devices

Pseudo-devices behave much like devices, but have no real hardware attached to them. Pseudo-devices are frequently abstractions that can be opened, read from, written to, and closed in the same way as real hardware.

For example, the loopback interface is a pseudo-device used for network connections to the local machine. (Your computer has no loopback network card, but the loopback interface behaves just like a real network card with an unusual MTU value.)

Pseudo-devices are labeled with `pseudo-device`.

```
pseudo-device   loop    # network loopback
pseudo-device   pf      # packet filter
pseudo-device   gre     # GRE encapsulation interface
```

Keywords

Finally, a handful of other keywords appear only once or rarely. These one-offs change how the kernel runs or how it's built, and defy easy categorization. The following keywords may appear:

- The `machine` keyword tells the kernel which architecture it should run on.
- The `makeoptions` keyword tells the compiler how to build the kernel.
- The `include` keyword means pull in another configuration file.
- The `maxusers` value sets the size of some in-kernel tables.

You'll find even less common keywords scattered in different kernel configurations.

```
machine         amd64
makeoptions     DEBUG="-g"  # compile full symbol table
include         "../../../conf/GENERIC"
maxusers        80          # estimated number of users
```

All of these affect the kernel in wildly different ways. You'll find several of these keywords in any kernel, even GENERIC.

Configuring GENERIC

Let's look at an actual kernel configuration. OpenBSD divides kernel configuration into machine-independent and machine-dependent files.

Machine-Independent Configuration

The machine-independent kernel configuration files are in */usr/src/sys/conf*. The file */usr/src/sys/conf/GENERIC* contains the machine-independent kernel configuration, which describes all of the features that OpenBSD supports on all hardware platforms. Every GENERIC kernel contains the configuration in this file. If you change this file, it will affect every kernel built that includes this file.

The machine-independent configuration file doesn't contain device drivers; instead, devices are tied to particular hardware. This file won't contain any special building instructions, because they vary from platform to platform. Nor will it include hard-coded system limits, data structure sizes, and so on, as OpenBSD running on a 25-year-old VAX has considerably fewer resources than a brand-new amd64 system. The */usr/src/sys/conf/GENERIC* file contains mostly options and pseudo-devices. Every OpenBSD kernel must support a filesystem, or it won't be able to write to disk or anything disk-like.

A kernel based on this file doesn't yet know what sort of hardware the filesystem will run on, but it knows how to make a filesystem. It doesn't know what kind of network card it will have, but once you give it a network card, it can create a TCP data stream and serve your web pages. You'll need the machine-dependent configuration to make a kernel that can function in the real world.

Machine-Dependent Configuration

Each platform has its own machine-dependent kernel directory under */usr/src/sys/arch*. Here's where you'll find a subdirectory for every platform OpenBSD supports, as well as a directory for any platforms under development. Separate directories contain platform-specific code, as well as further *conf* subdirectories for the kernel configuration file.

I'm using amd64 as an example, so the kernel configuration directory is */usr/src/sys/arch/amd64/conf*. While we'll focus on the common i386 and amd64 architectures, the kernel-building process is the same across all hardware platforms.

A traditional kernel configuration filename is in all capital letters. You'll see the *GENERIC* configuration, as well as the *RAMDISK** files used for the installation disks. (The *GENERIC.MP* kernel is the multiprocessor kernel.) We'll start with the *GENERIC* kernel configuration file:

```
machine         amd64
include         "../../../conf/GENERIC"
```

The first entry in this kernel configuration defines the machine. The machine definition tells the kernel configuration parser the kind of hardware you're running, and defines core hardware characteristics and constraints, such as how many bits are in an integer and how much memory the system can support.

The second entry pulls in the machine-independent kernel configuration (described in the previous section), defining all of the protocols and tools that make OpenBSD OpenBSD. The amd64 kernel inherits the filesystems and network stacks from this entry.

Following these two lines you'll see the devices OpenBSD supports on amd64 hardware. Take a moment and skim the file. It's the same mix of devices and attachments as described earlier in this chapter.

Your Kernel Configuration

In order to build your own kernel, you'll need a configuration file. Here, we'll look at how to create your configuration file. (Do not just edit either GENERIC kernel file.)

Minor Changes

If your kernel adds only a couple of items to the GENERIC kernel, use the GENERIC configuration as a basis for your new one. For example, here's the multiprocessor kernel configuration, GENERIC.MP:

```
include  "arch/amd64/conf/GENERIC"
option          MULTIPROCESSOR  # Multiple processor support
cpu*            at mainbus?
```

The multiprocessor kernel builds on GENERIC, adding only one option and one device attachment. You can use this model to define your own kernel configuration.

For example, suppose you want to enable the experimental SCSI multipathing feature on a kernel. You could create a kernel configuration file in your platform directory, and simply copy the commented-out multipathing entries from the machine-independent GENERIC kernel, like this:

```
include    "arch/amd64/conf/GENERIC"
mpath0     at root
scsibus*   at mpath?
```

This creates a custom kernel that closely resembles GENERIC, with these two extra devices.

Removing Options

To strip options from your kernel, use the `rmoption` keyword. For example, to create a minimal kernel based on GENERIC, you could use the `rmoption` keyword to remove some kernel options, as in this example:

```
include "arch/amd64/conf/GENERIC"
rmoption    NTFS
rmoption    HIBERNATE
...
```

One advantage to creating a configuration by including the default kernel is that when you update your source code, your custom kernel configuration will probably still be valid. However, the more options you remove from the kernel, the greater the chance that the kernel will fail to compile, or if it compiles, that it might not boot. And if it boots, it might eat your hard drive.

When removing options, keep in mind that some options are more important than you might think. For example, removing the `INET6` option (aka IPv6) can create a nonfunctional system. Removing options doesn't save you much memory, and it might cripple any number of programs.

Removing Devices

If you want to remove a lot of stuff from a machine-dependent kernel configuration, while retaining the options for base OpenBSD functions, copy the machine-dependent GENERIC configuration file to a new text file and make your changes in that file.

Wholesale Butchery

If you want to commit wholesale butchery on the kernel, you'll want a configuration that includes both the machine-independent and machine-dependent parts. Start by copying the existing GENERIC kernel configurations into one file, in the platform's kernel configuration. Here, I call my new kernel TREBLE, after the hostname:

```
# cd /usr/src/sys/arch/amd64/conf
# cp ../../../conf/GENERIC TREBLE
# cat GENERIC >> TREBLE
```

Before making any other changes, remove the line that includes the machine-independent kernel configuration file. Then slice out everything that makes the system functional, and try to build the new kernel. Next, add stuff back in until the kernel builds. (Although removing drivers won't save much memory, doing so will make booting a tiny bit faster.)

NOTE *You might be tempted to use the man pages to create your own kernel configuration from scratch. You're certainly free to do that, if you're either a Kernel Lord or an irremediable doofus. Feel free to try it. Every sysadmin can use such a valuable lesson in humility.*

Stripping Down the Kernel

Every device driver and option in the kernel uses memory. If you're trying to cram OpenBSD onto a tiny computer, or you're doing any sort of embedded development, you might want to build a custom kernel that includes as few device drivers as possible by editing */var/run/dmesg.boot*, where every entry matches a line in the kernel configuration.

The simplest way to trim out unnecessary device drivers is to remove everything that's not in your computer. The kernel includes dozens of network card drivers, but you need only one or two. If you're unsure about a device, keep it in the configuration. (The ACPI and BIOS devices in particular are tightly interrelated, and you'll probably have a really hard time building a bootable custom kernel without the complete set of ACPI and BIOS devices.)

Gutting the Kernel

If removing device drivers doesn't create a sufficiently small kernel for you, try removing machine-independent options. Many of these options are interdependent, however, and removing them can create a kernel you can't compile. If you can compile the kernel, it might not boot, and if it boots, it might not function correctly.

Testing Your Kernel Configuration with config(8)

Is your custom kernel configuration internally consistent? To test your kernel and prepare the files needed to compile it, use `config(8)`.

While still in the kernel configuration directory, give `config` the kernel configuration filename as an argument, like this:

```
# config TREBLE
```

If you get any error messages, read them. For example, `config` might tell you that you need to run `make clean` before building your new kernel, or that your kernel configuration is internally inconsistent and will not compile. If there's a problem, `config` will often give a line number where you made an error. Follow any advice `config` offers.

The following are some of the more common types of errors.

Orphaned Devices

One common way that `config` fails is if you're missing a device that's needed by another device. Here's an example:

```
# config TREBLE
TREBLE:36: cpu0 at mainbus? is orphaned
 (nothing matching mainbus? declared)
TREBLE:37: bios0 at mainbus0 is orphaned
 (no mainbus0 declared)
TREBLE:38: ioapic* at mainbus? is orphaned
 (nothing matching mainbus? declared)
TREBLE:82: pci* at mainbus0 is orphaned
 (no mainbus0 declared)
*** Stop.
```

Your configuration attaches various devices to `mainbus0`, but there's no `mainbus0` entry in your configuration. Kernels that include devices that aren't attached don't make sense and cannot compile.

To address this, examine your hardware again. Figure out how these devices are supposed to attach to the system, and fix your kernel configuration.

Bogus Hardware

Another common problem is including nonexistent device drivers, which generates the following error.

```
# config TREBLE
TREBLE:36: cpe0: unknown device `cpe'
*** Stop.
```

`config` shows me the error and the line number where it occurs. There is no `cpe` device, but there is a `cpu` device. My bad.

The error checking performed by `config` does not guarantee that your kernel will compile or run as expected. The only errors it catches are ones where the configuration is either internally inconsistent or flat-out wrong. The first real test comes when you try to actually build your configured kernel.

Building a Kernel

If `config` ran successfully, you will have a kernel compilation directory including a makefile and a whole slew of header files. The traditional place for the *compile* directory is under the platform directory, which is */usr/src/sys/arch/amd64* for amd64 hardware.

The compile directory contains a subdirectory for each kernel configuration processed by `config`. My amd64 kernel called TREBLE is in the */usr/src/sys/arch/amd64/compile/TREBLE* directory, which contains a makefile, as well as all the header files for all included devices and options.

```
# cd ../compile/TREBLE
# make
```

Now it's time to wait. A successful compilation will create a kernel file *bsd* without generating any error messages.

Kernel Build Errors

If your kernel fails to build, you probably have a perfectly explicable error. First, read the error message given by the compilation. Most of the time, the error message will explain what the kernel is missing. Generally, you will need to change your kernel configuration in some manner because of an error that `config` could not catch. A broken kernel compilation will end something like this:

```
../../../../arch/amd64/pci/pci_machdep.c: In function ❶'pci_intr_map':
../../../../arch/amd64/pci/pci_machdep.c:641: error: ❷'PCI_INT_VIA_ISA'
undeclared (first use in this function)
../../../../arch/amd64/pci/pci_machdep.c:641: error: (Each undeclared
identifier is reported only once
❸ ../../../../arch/amd64/pci/pci_machdep.c:641: error: for each function it
appears in.)
*** Error code 1

❹ Stop in /usr/src/sys/arch/amd64/compile/ENVY (line 89 of /usr/share/mk/sys.mk).
```

This kernel cannot be built because something is missing. When a build fails with statements that something "is undeclared" (as shown in bold), that's a hint that the kernel is missing a necessary entry.

The name of where it failed might offer you a hint as to what's missing. In this case, at ❶, I have a function name where the compilation failed, and then a specific undeclared variable ❷ that caused the compilation to fail.

I would start by figuring out where the `pci_intr_map` function comes from and what it's supposed to do. Search the source code and man pages for references to the missing function. Failing that, try the mailing list archives. Be sure to include the function and variable names in any web search. Generic output that says that "there was an error" ❸ or "the compile has stopped" ❹ is less unique, and hence it could be useful. If all else fails, fall back to the GENERIC configuration.

Installing Your Kernel

Your completed kernel is the file *bsd* in the compile directory. Before you use your new kernel, verify that you have your current, working, well-behaved kernel backed up to a separate file on the root filesystem, and then copy your new kernel to */bsd*. That's it! The next time you reboot, you'll come up on your new kernel.

NOTE *Some people do not like to copy their custom kernel to /*bsd *until they're certain that the kernel will boot. If you're one of these people, copy your new kernel to the root directory under a different name, such as /*bsd.test. *Boot into this alternate kernel. Test your system. If everything works, properly install your new kernel.*

Identifying the Running Kernel

If you build several custom kernels, you might forget which kernel you're running. The `uname(1)` command will tell you the name of the kernel configuration file used to build the running kernel. The `-v` flag will tell you the name of your kernel configuration and the number of times you have compiled it.

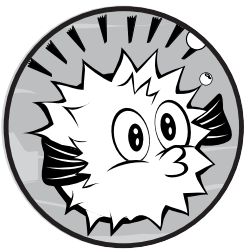
```
# uname -v
GENERIC.MP#348
```

This output does not mean that I've built a multiprocessor GENERIC kernel 348 times. I use the GENERIC kernel, and I let the OpenBSD release engineers build my kernels for me. They have built 348 official snapshot multiprocessor kernels without wiping the kernel build directory. Remember that building custom kernels is for advanced programmers and ignorant newbies. I'm neither.

20

UPGRADING

The latest source code?
Fugu extraordinary!
Be brave and swallow.



Here's an ugly truth: If upgrades are hard, sysadmins try their best to avoid them. And that can cause security problems. Operating system upgrades can cause software that has worked well for years to develop nervous tics or stop working altogether. Fixing add-on packages that don't work on the new operating system version can require days of troubleshooting. Server upgrades can make even seasoned sysadmins wish that they had a simpler job, such as performing as a carnival sideshow, stuffing weasels into their trousers.

While you can probably deal with a bit of odd behavior in a desktop after an upgrade, your servers and firewalls must behave exactly as expected. It's common to delay upgrades until the system is so old that it can be replaced with a new machine running the new release, but that's both terrible system administration practice and completely unacceptable security practice. Computers connected to the Internet must be patched, maintained, and upgraded, or an intruder will almost certainly compromise them.

Fortunately, the OpenBSD upgrade process is simpler than those used by many other Unix-like operating systems. With proper preparation, you can upgrade OpenBSD with a minimum of difficulty.

Why Upgrade?

Because you don't have a choice.

Security researchers, programmers, and skilled intruders continuously discover new ways to penetrate previously secure systems. Although OpenBSD has suffered only two vulnerabilities in a default installation that permitted an intruder to compromise the system, that doesn't mean that a two-year-old OpenBSD version is secure.

The OpenBSD Project provides security updates for only the two most recent releases. For example, when OpenBSD 5.3 comes out, OpenBSD 5.1 will be "end-of-lifed" and lose support from developers. If someone figures out how to break into a default OpenBSD 5.1 installation after 5.3 comes out, the developers might not provide fixes. You might adjust new security patches to work on older versions of the code, but you will find that backporting fixes becomes increasingly difficult.

Software not enabled in the default installation can also have problems. For example, in November 2011, the OpenBSD Project released a patch for a problem in the included BIND `named(8)` name server. OpenBSD does not ship with `named` enabled, so this wasn't a problem in the default installation, but it was certainly a security problem in a function that you might have chosen to enable.

In order to protect your system, you must understand how to apply security patches, either by applying the patch or by upgrading the entire system. But before we get to the upgrade process itself, let's look at your choices for OpenBSD versions.

OpenBSD Versions

Developers all over the world continually make minor changes to OpenBSD's source code. If you download the source code in the morning and again in the afternoon, you'll get two slightly different versions. In fact, at any given time, you can get a few different versions of OpenBSD: *-current*, *snapshots*, *releases*, and *-stable*.

OpenBSD-current

OpenBSD-current is the most recent development version of OpenBSD, containing code that is making its public debut. Developers test each other's work and then get approval to commit code to the tree. Eventually, they must put their work out for the public to test, review, and debug.

OpenBSD-current is where the public can access the newest code. If a change would temporarily break web servers, games, database servers, or whatever is running on -current, but the change is for the long-term good,

the change will go into -current. Those programs will probably work again before the next OpenBSD release, but there's no requirement that every third-party program work perfectly at all times on -current.

NOTE *The developers expect -current itself to work at all times, and they consider breaks serious problems. You might need to recompile Firefox, or the port maintainer might need to rewrite a makefile, but the core operating system is expected to run at all times.*

With these caveats, why would you possibly want to run -current? One excellent reason is to test the new OpenBSD in your environment. If today's -current panics under certain conditions, but last week's didn't, the OpenBSD folks want to know that.

(All the OpenBSD folks run -current on their laptops, and they run firewalls on -current.) If you can exercise -current in the real world, with a real workload, they want you to do so. Having real users run -current is the only way that OpenBSD can be tested before the official releases come out.

OpenBSD Snapshots

Every few days, the OpenBSD team uploads a release from the latest -current code to the mirror servers. This is an interim release called a *snapshot*, which is identified only by the date of its release. A snapshot is simply the state of -current at a particular time. While the developers try not to build a snapshot on a day that -current is notably fouled up, guess who's in charge of the snapshot quality-assurance process. That's you.

Snapshots are provided for installation and testing convenience. Installing and testing a snapshot is easier than building -current on your own. Also, if you can reproduce a bug on a specific snapshot, the developers know exactly which version of the code you are running. It's much easier to identify a problem as appeared "in the July 15 2013 snapshot" than in a -current you built yourself from "code downloaded from server X at about 3:17 PM EST on July 15, 2013."

NOTE *Snapshots can contain uncommitted code not in -current. The patches for the changes are not usually available.*

You can get snapshots installation media from any mirror site, in the */pub/OpenBSD/snapshots* directory. If you want to run -current, the recommended way to start is by installing the most recent snapshot you can get your hands on and upgrading from there.

OpenBSD Releases

Every six months, the pace of OpenBSD development is deliberately slowed. Developers spend time polishing new features, and Theo makes increasingly forceful requests for beta testers of the newest snapshots. When the OpenBSD team is satisfied that the software is of adequate quality, the source code tree is tagged, and a high-quality snapshot is built. This

snapshot is called a *release* and is issued a number like 5.2, 5.3, and so on. This is almost certainly what you installed on your first OpenBSD machine. A new release appears every May and November.

OpenBSD numbers releases sequentially, starting with 2.0 and incrementing .1 with every release. Unlike most software products, a .0 release has no special meaning. It's just another spot along a long path—a milestone hit every five years.

The release is the best-supported version of OpenBSD. Everything is expected to work, and the development team stands behind its releases. If a serious security problem is discovered in a release, OpenBSD will release a patch, or *errata*, for it.

OpenBSD-stable

OpenBSD-stable is an OpenBSD release with all errata and very minor patches included. The developers deliberately hold the number of changes to each -stable version to the absolute bare minimum. A stable version is referred to as its base release plus -stable, such as 5.4-stable, 5.5-stable, and so on.

What sort of changes are merged back into -stable? Security fixes and errata go in automatically. Other than security issues, any patches added to -stable must be simple, short, and obviously correct. Patches that dramatically affect a small number of users might go in. For example, if all systems with a specific network card panic at random intervals, a patch might go into -stable (accompanied by various developers asking why no one with that network card ran snapshots before the release).

You won't get new functions in -stable. It won't come with any new device drivers or packet-filtering features. The API will not change. In general, -stable is expected to never get worse.

The only way to get OpenBSD-stable is to update the system from source.

Which Version Should You Use?

OpenBSD's release system combines the best features of open source development and commercial releases. Users have access to both the bleeding-edge experimental code and the stable, polished releases. Look at your requirements and choose your poison.

- If you're running OpenBSD in a production environment, either use a release with the applicable security errata or track -stable. The developers encourage more experienced users to use snapshots or -current, but that choice is up to you.
- If you are evaluating OpenBSD for use in a production environment, install the version you intend to use.
- If you're just learning about Unix-like systems, or if you want a quiet OpenBSD experience, use a release and apply applicable errata.

- If you're an operating system developer or experienced sysadmin, feel free to jump right in to -current or snapshots. You can either handle any problems you encounter or use those problems as an excuse to expand your knowledge. Many people find that they can use -current in production, if they do not need packages. These are usually more experienced users who want firewalls.
- Hobbyists can run anything they want!

Remember the limitations of your chosen branch. A release is a good place to start, but you can gradually upgrade your system to -stable, and then to -current, as your understanding expands. Many developers started out as interested hobbyists.

The OpenBSD Upgrade Process

An OpenBSD upgrade has three distinct phases: installing the newer versions of the operating system files, updating the local configuration, and updating obsolete add-on software packages. Each is a separate part that requires independent handling.

An upgrade requires installation media. The best upgrade media is the new release CD or an Internet mirror. You can also upgrade OpenBSD by building and installing the source code directly on the machine to be upgraded, but doing so is more difficult and risky than upgrading from the official release. Much of the information for upgrading from a network or CD applies to upgrading via source code.

NOTE *OpenBSD supports upgrading only one major release at a time. You cannot upgrade directly from, say, 5.3 to 5.5; you must upgrade from 5.3 to 5.4, and then to 5.5. The more releases you need to upgrade through, the more reason to reinstall. You would spend more time serially upgrading three or four releases than you would reinstalling.*

Before upgrading, back up any data you actually care about. The upgrade process extracts new files over the existing operating system, and could overwrite something important. The installer generally works, but human beings are fallible. Back up!

Following the Upgrade Guide

As OpenBSD evolves, basic system features change. This wouldn't be a big issue, except when interdependent changes create a chicken-and-egg or bootstrap problem. If you just blindly run the upgrade process and don't handle any other required changes, you'll find that your new system fails in unpredictable ways.[1] If you're building the system from source, these problems might prevent the build from completing.

1. Henning Brauer tells me that many upgrade failures aren't really unpredictable; they're merely "unsupported and untested" code paths. To most of us, that's "unpredictable," but you're welcome to predict them yourself.

All of these potential problems should be solvable by anyone building the system from source, but it's nice to have them documented. Conveniently, OpenBSD provides an *Upgrade Guide* for each release, documenting the steps needed to take a system from one release to the next.

The *Upgrade Guide* is divided into chunks by the type of upgrade you are performing. The simplest upgrade is for people using official OpenBSD files, such as a network or CD upgrade. If you're building from source, the instructions quickly grow in complexity. Follow the instructions in the order in which they appear. Most upgrade prerequisites are typical system administration tasks. The following are the most common requirements.

NOTE *Instructions in the* Upgrade Guide *supersede anything I say in this chapter. I could just sprinkle the words "unless specified otherwise in the* Upgrade Guide*" in every paragraph for the rest of this section, but then my editor would slap me. The OpenBSD documentation is the final word in OpenBSD system administration, including the upgrade process.*

Install Programs

Especially when you are building from source, bootstrap tools such as `gcc(1)` and `perl(1)` might need to be built with the upgraded tool. When upgrading from source, you might need to install these bootstrap tools before beginning to compile the new version of OpenBSD. The *Upgrade Guide* notes these requirements.

If your attempt to build OpenBSD from source fails, reread the *Upgrade Guide* before troubleshooting. Starting from the closest available binary release or snapshot will probably solve your problem.

Remove Programs and Files

OpenBSD removes programs from the base system when they become obsolete or dangerous, or when their functions are integrated into other programs. When you upgrade, you must manually remove these programs. This is necessary because old software left on an upgraded system can pose a security risk. Also, some files and directories might become superfluous in a new OpenBSD version. You can remove these programs, files, or directories once the upgrade is complete.

Prepare Package Upgrades

When you upgrade OpenBSD, you must upgrade the installed packages as well, because you can't reliably run packages built for old versions of OpenBSD on newer versions. (Doing so *might* work, but there's no guarantee.) The *Upgrade Guide* includes notes on upgrading specific installed packages, and you should take some of those actions before beginning the upgrade.

For example, the *Upgrade Guide* for the newest OpenBSD version says that the PostgreSQL port had a major version upgrade. You must do a database

dump and restore as part of the upgrade. The old version of PostgreSQL might not run well on the new OpenBSD, so perform the database dump before starting the system upgrade.

Your packages might not require any pre-upgrade work, but it's much easier to check beforehand than to finish the operating system upgrade, and then need to fall back because you didn't prepare some third-party package.

System Configuration

The preceding tasks are the most common requirements for upgrading, but you might need to change other system files before or after the upgrade process itself. Read the *Upgrade Guide*, or your programs might not run as expected.

Customizing Upgrades

The upgrade script supports the *siteXX.tgz* file discussed in Chapter 23. If the file exists in the installation media, you can choose to install it during the upgrade.

You can also run a custom post-upgrade script as part of the upgrade. When the upgrade completes, the script checks for the file */upgrade.site*. If the upgrader finds this file, it executes this script as the last step of the upgrade. Copy */upgrade.site* to the system before starting the upgrade. Chapter 23 discusses *upgrade.site* in more detail.

Upgrading from Official Media

After reading the *Upgrade Guide*, get your installation media for the new version of OpenBSD and boot from it. If you plan to upgrade over a network, you should need only the new installation kernel *bsd.rd*. You can just grab this via FTP (or you could grab the entire directory along with it and run the upgrade from your local disk). Here, I grab the newest snapshot kernel from my root directory:

```
# cd /
# ftp ftp://ftp3.usa.openbsd.org/pub/OpenBSD/snapshots/amd64/bsd.rd
Connected to plier.ucar.edu.
...
```

After you get the snapshot kernel, go to your console and boot into the new kernel.

```
>> OpenBSD/amd64 BOOT 3.18
boot>  boot bsd.rd
booting hd0a:bsd.rd: 2993636+916428+2864232+0+531344
[89+320016+207017]=0xb799f0
entry point at 0x1001e0 [7205c766, 34000004, 24448b12, a608a304]
...
```

```
root on rd0a swap on rd0b dump on rd0b
erase ^?, werase ^W, kill ^U, intr ^C, status ^T

Welcome to the OpenBSD/amd64 5.3 installation program.
(I)nstall, (U)pgrade or (S)hell? U
```

Enter `U` to upgrade. The upgrade process looks much like the installation process covered in Chapter 3. Defaults appear inside square brackets. Accept the defaults by pressing ENTER.

```
Terminal type? [vt220] ❶
Available disks are: sd0 sd1 sd2.
Which one is the root disk? (or 'done') [sd0] ❷
Root filesystem? [sd0a] ❸
Checking root filesystem (fsck -fp /dev/sd0a)...OK.
Mounting root filesystem (mount -o ro /dev/sd0a /mnt)...OK.
Do you want to do any manual network configuration? [no] ❹
Force checking of clean non-root filesystems? [no] ❺
fsck -p e4bf0318329fe596.a...OK.
...
```

Always use the default terminal, unless you know exactly when you shouldn't. Commodity hardware usually uses vt220, as shown here at ❶, but the default terminal is platform-specific.

The upgrade needs to read your root partition to learn where to install files. It can't conclusively identify your actual root disk ❷ and partition ❸ unless you tell it to do that.

The upgrade script will configure your network according to the existing settings that you should hope are correct. If you need to adjust the network at every boot, the upgrade script gives you a chance to reconfigure the network at ❹.

If you shut down the machine via `reboot` or `shutdown`, your filesystems should be clean. If you unceremoniously pulled the power plug because you were going to upgrade the machine and no longer cared about your filesystems, OpenBSD will notice and clean your filesystems. To deep-check clean filesystems, you can force running `fsck(8)` at ❺. The upgrade script preens clean filesystems to check for obvious errors before proceeding.

Upgrading Over the Network

The default upgrade method is CD. I want to do this upgrade over the network, so here's how I continue:

```
Location of sets? (cd disk ftp http or 'done') [cd] ftp ❶
HTTP/FTP proxy URL? (e.g. 'http://proxy:8080', or 'none') [none] ❷
Server? (hostname, list#, 'done' or '?') [ftp5.usa.openbsd.org] ftp3.usa.openbsd.org ❸
Server directory? [pub/OpenBSD/snapshots/amd64] ❹
Login? [anonymous] ❺
```

I choose `ftp` as the location of the sets at ❶. I don't need to go through a proxy server to access the FTP server, so I leave that space blank at ❷.

The default OpenBSD FTP server is perfectly fine, but if you've identified a really fast mirror, you might use that. I use my preferred mirror site at ❸.

Every *bsd.rd* installer knows the server directory to install from at ❹. I change the server directory only if I have set up a local mirror. Similarly, every OpenBSD mirror permits anonymous FTP. I change the username and enter a password only if I'm using a local mirror at ❺.

Choosing File Sets

Next comes a chance to choose which sets to upgrade.

```
Select sets by entering a set name, a file name pattern or 'all'. De-select
sets by prepending a '-' to the set name, file name pattern or 'all'. Selected
sets are labelled '[X]'.
    [X] bsd           [X] base51.tgz    [X] game51.tgz    [X] xfont51.tgz
    [X] bsd.rd        [X] comp51.tgz    [X] xbase51.tgz   [X] xserv51.tgz
    [X] bsd.mp        [X] man51.tgz     [X] xshare51.tgz
Set name(s)? (or 'abort' or 'done') [done]
```

You must upgrade every file set installed on your machine, or the machine will behave unpredictably. If you didn't install some sets during your original installation, you don't need to install them now. For most machines, I recommend installing all sets.

NOTE *Notice that two sets are missing:* etcXX.tgz *and* xetcXX.tgz. *These files belong in* /etc *and are legitimately edited by system administrators. The upgrade script cannot know if a file should be replaced, edited, or ignored. You must update* /etc *yourself.*

The upgrade script downloads and extracts the selected file sets, and then asks you to verify that you're finished selecting file sets. If so, it remakes all your device nodes to fit with the new kernel.

At this point, you can reboot into your new OpenBSD userland except that userland might not work quite right because you haven't updated */etc* yet.

Updating /etc

The */etc* directory contains system and program configuration information. When a program changes, its configuration file might also change. If you try to run a new program with an obsolete configuration file, the program will not run correctly.

You absolutely must update */etc* before running your system—arguably the most annoying part of an upgrade. No automated process can possible know how your system should behave. Only you can know that, which means that you must compare the contents of your existing */etc* to the same files in a new, stock */etc*. OpenBSD provides tools to make the process less annoying.

Before you begin, if your system performs complicated functions, such as a database or packet filtering, boot into single-user mode. (Although

single-user mode is not strictly necessary, I've had software behave badly during a half-completed upgrade, so I now routinely update */etc* in single-user mode.)

```
 >> OpenBSD/amd64 BOOT 3.19
boot> boot -s
booting hd0a:/bsd: 5687720+1608588+939456+0+644288 [80+502320+325975]=0xd43b98
...
Enter pathname of shell or RETURN for sh:
#
```

While I'm a long-time `tcsh` user,[2] I always use the default shell in single-user mode. If you want to use an alternate shell, it must be statically linked and available on the root partition.

Mounting Filesystems

Now mount all your filesystems. If you upgraded via the network, start the network now.

```
# mount -a
# /bin/sh /etc/netstart
WARNING: inconsistent config - check /etc/sysctl.conf for IPv6 autoconf
#
```

I've hardly begun, and here's a warning already. How fantastic! Perhaps this IPv6 error is from the upgrade, or maybe it's a lingering misconfiguration I never noticed before. Either way, this is a good time to identify and fix the problem.

Verify that you're on the network by pinging a couple of hosts.

You need the files *etcXX.tgz* and *xetcXX.tgz* for your new release. If you have the CD, the files are in the release directory. If not, fetch them over the network. (I always get these file sets from the same server I upgraded from, just to eliminate any chance of a difference in the files.)

```
# cd /tmp
# ftp ftp://ftp3.usa.openbsd.org/pub/OpenBSD/snapshots/amd64/etc51.tgz
# ftp ftp://ftp3.usa.openbsd.org/pub/OpenBSD/snapshots/amd64/xetc51.tgz
```

You are now ready to update */etc*.

Using sysmerge(8) to Compare /etc Files

The `sysmerge(8)` program compares your existing */etc* to the */etc* in the installation set, points out differences, and lets you replace your installed file with the new one, keep your file, or merge the two.

2. I know, I know, your shell is superior to mine. I was given `tcsh` as my first shell almost 30 years ago, and my fingers are too habituated to it to change. I'll concede your superiority if you'll stop telling me about it.

Use -s to tell sysmerge where the new *etcXX.tgz* file is, and -x to point to the new *xetcXX.tgz*. I put both of these files in */tmp*.

```
# sysmerge -s /tmp/etc51.tgz -x /tmp/xetc51.tgz
```

sysmerge will handle the easy changes itself, but leave you to take care of files that need your intervention.

Easy sysmerge Updates

If your system is only lightly modified, you should see something like this:

```
===> Populating temporary root under /var/tmp/sysmerge.daiHKukKfE/temproot
===> Starting comparison
===> Updating /etc/inetd.conf
===> Updating /etc/login.conf
...
```

I haven't changed these files, so sysmerge automatically updates them for me.

```
...
===> Installing /etc/nginx/fastcgi_params
===> Installing /etc/nginx/koi-utf
...
```

I didn't have these files on my system, so sysmerge installs them for me.

These examples are easy cases. In cases where you've edited an */etc* file, sysmerge will need your help.

sysmerge and Edited Files

Say you've edited a file, and the file version has changed. Does your local change reflect the update, does it conflict, or can the two coexist? sysmerge can't tell if it should overwrite the installed file with a new version, leave it unchanged, or merge the two, so it displays the differences between the old and new files.

```
===> Displaying differences between ./etc/mail/aliases and installed version:

❶ --- /etc/mail/aliases   Sun Jun  9 04:50:04 2013
❷ +++ ./etc/mail/aliases  Wed Oct 23 17:06:02 2013
@@ -1,5 +1,5 @@
 #
❸ -#      $OpenBSD: aliases,v 1.36 2010/09/22 13:01:10 deraadt Exp $
❹ +#      $OpenBSD: aliases,v 1.37 2012/10/13 07:42:39 dcoppa Exp $
 #
 #  Aliases in this file will NOT be expanded in the header from
 #  Mail, but WILL be visible over networks or from /usr/libexec/mail.local.
@@ -32,6 +32,7 @@
 _identd: /dev/null
 _iked: /dev/null
```

```
    _isakmpd: /dev/null
❺ +_iscsid: /dev/null
    _kadmin: /dev/null
    _kdc: /dev/null
    _ldapd: /dev/null
   @@ -69,7 +70,7 @@
    sshd:   /dev/null

    # Well-known aliases -- these should be filled in!
❻ -root: mwlucas@blackhelicopters.org
❼ +# root:
    # manager:
    # dumper:
```

If you've changed the file, `sysmerge` presents the differences between the old and new files with a few surrounding lines of context. In this example, the */etc/mail/aliases* file on my system is dated June 9 at ❶, and the new version is from October 23 at ❷. Lines that exist in the new version of the file but not in the installed one are prefaced with a plus sign (+). Lines that exist in the installed file but not in the new one are prefaced with a minus sign (-).

This example shows that the OpenBSD version of this file changed from 1.36 at ❸ to 1.37 at ❹. That's not terribly surprising, and for system administration purposes, not a terribly vital piece of information. But the new version has a new alias, directing email addressed to the user `_iscsid` to */dev/null* at ❺. This could be important, so I want the new alias on my system.

Next, we see a difference between the files, where I've forwarded email addressed to root to my personal email at ❻, but the new version of the file has no such redirection at ❼. I want the old version of this line but the new version of another line, and while I don't particularly care about the OpenBSD version numbers, I would prefer the newer one. I could hand assemble an aliases file, but after `sysmerge` prints out the differences, it offers to help.

```
❶ Use 'd' to delete the temporary ./etc/mail/aliases
❷ Use 'i' to install the temporary ./etc/mail/aliases
❸ Use 'm' to merge the temporary and installed versions
❹ Use 'v' to view the diff results again

  Default is to leave the temporary file to deal with by hand

How should I deal with this? [Leave it for later] m
```

`sysmerge` offers the following choices:

- If I delete the temporary aliases file, `sysmerge` throws away the new file ❶. This is fine if my existing aliases file is adequate, but in this case, I want part of the new file.
- If I install the temporary aliases file, I overwrite my changes ❷. Unless I go back in and change the aliases file again, my custom mail forwarding will stop working. I don't want this to happen.

- I can merge the old and new files ❸, choosing the best fit from each file, which is the option I'll pick.
- I could look at the differences again ❹. This has the advantage of putting off an actual decision.

To merge the old and new files, I enter `m`. Note that the merge function requires close attention to detail. A mistake here can make your system hang as it tries to go into multiuser mode.

When you merge, `sysmerge` takes your input and builds a combined file. Lines that are identical in both versions are automatically added to the new file. When lines differ, `sysmerge` walks you through the differences and lets you choose a version.

The new version of the line appears on the right, and the installed version is on the left. Enter **`l`** or **`1`** to choose the version on the left side, or **`r`** or **`2`** to choose the version on the right.

```
# $OpenBSD: aliases,v 1.36 2010/09/22 13:01:10 de | # $OpenBSD: aliases,v 1.37 2012/10/13
07:42:39 dc
%r
```

The version number has updated, though it's in a comment, which won't affect how the alias file updates. Still, as I'm merging the files anyway, I might as well get the new version number. I enter **`r`** to choose the version on the right.

```
                                > _iscsid: /dev/null
%r
```

There is no equivalent to this line in the installed file, as this alias exists only in the new file. Again, I choose **`r`** to include this line in my file.

```
root: mwlucas@blackhelicopters.org    | # root:
%l
```

Here, I want the entry from the installed file, on the left-hand side. I choose **`l`**.

Once I've gone through all of the entries, `sysmerge` offers me more choices. These include comparing the merged and installed files, comparing the merged and new files, starting over, viewing the merged file, redoing the merge, and installing the merged file.

```
  Use 'e' to edit the merged file
  Use 'i' to install the merged file
  Use 'n' to view a diff between the merged and new files
  Use 'o' to view a diff between the old and merged files
  Use 'r' to re-do the merge
  Use 'v' to view the merged file
  Use 'x' to delete the merged file and go back to previous menu
  Default is to leave the temporary file to deal with by hand
===> How should I deal with the merged file? [Leave it for later] i
```

I did everything correctly, so I install the merged file (although I should probably view the merged file first, and then install it).

My biggest difficulty with `sysmerge` comes in differentiating my left and my right. Worse, the L key is on the right side, and the R key is on the left. (Go ahead now, laugh, but just wait until you mix that up.)

Finishing sysmerge

When `sysmerge` finishes installing the files, it checks the permissions on the */etc* files, and tells you where to find the log and that the system might need a reboot. A purist might tell you that you don't need to reboot, but usually you should go ahead and do it. Rebooting after changing core system files prevents a variety of problems, and rebooting as part of an upgrade isn't unreasonable. You can wait to reboot until after you upgrade your packages, as long as everything works.

```
...
===> Comparison complete
===> Checking directory hierarchy permissions (running mtree(8))
===> Output log available at /var/tmp/sysmerge.daiHKukKfE/sysmerge.log
        *** WARNING: some new/updated file(s) may require a reboot
# reboot
```

You now have an updated OpenBSD base system. Be aware that new or changed files can change system behavior, but usually you won't notice. The *Upgrade Guide* usually notes any changes average users are likely to notice. Still, only you know what your system does and how you want it to behave.

Upgrading your installed packages is a separate task.

Updating Installed Packages

Packages run reliably only on the version of OpenBSD for which they're compiled. If you're using packages, then updating your third-party software is very easy. (If you build your own software from the ports collection, you can still update but it won't be as easy.)

First, check the *Upgrade Guide* again. It describes any intrusive changes to major software. Take any actions it recommends before continuing.

Updating the Package Repository

Before upgrading your packages, check your `$PKG_PATH` environment variable. It almost certainly references the package directory for your previous version of OpenBSD.

```
# echo $PKG_PATH
ftp://ftp11.usa.openbsd.org/pub/OpenBSD/5.2/packages/i386/
```

Find the package repository for your new version of OpenBSD. You can probably just update the release number in your shell's dotfile, but go to the mirror site and make sure that the packages for that release are present.

If you upgrade from release to release, you can use the `uname(1)` command to set your `PKG_PATH` in your dotfile. For example, if *ftp11.usa.openbsd.org* is your favorite mirror site, use a line like this for `sh`-based dotfiles.

```
export PKG_PATH=ftp://ftp11.usa.openbsd.org/pub/OpenBSD/`uname -r`/packages/`uname -m`/
```

Using the Upgrade Command

To upgrade your installed packages, use `pkg_add` with the `-i` and `-u` flags.

```
# pkg_add -iu
quirks-1.73->1.77: ok
apr-1.4.6->1.4.6p0: ok
apr-util-1.4.1-ldap:cyrus-sasl-2.1.25p3->2.1.25p3: ok
apr-util-1.4.1-ldap:openldap-client-2.4.31->2.4.31: ok
apr-util-1.4.1-ldap:libiconv-1.14->1.14: ok
...
```

The `-i` flag tells `pkg_add` to work in interactive mode and ask you about any ambiguities. The `-u` flag means "update."

This upgrader recurses through each of your add-on software packages and its dependencies, uninstalling the old version and installing the new. If you want to see more verbose and detailed messages about the package-updating process, add the `-v` flag.

Package Options

If the dependencies for a package have changed and you now have multiple options, `pkg_add` presents your choices.

```
Ambiguous: choose dependency for foomatic-db-engine-4.0.8p2:
 a       0: curl-7.26.0
         1: wget-1.13.4
Your choice: 1
```

The package `foomatic-db-engine` can use either `curl` or `wget`. Of the two, I prefer `wget`, so I enter `1`. Pressing ENTER tells `pkg_add` to use the default.

Package Messages

Once all of the packages have been updated, `pkg_add` displays any messages from the upgraded packages.

```
Read shared items: ok
You may wish to update your font path for /usr/local/share/ghostscript/fonts
Look in /usr/local/share/doc/pkg-readmes for extra documentation.
...
```

Some of the installation instructions will tell you to clear out cached files from the older version of the software, as in this CUPS example.

```
--- -cups-1.5.3p4 -------------------
You should also run rm -rf /var/log/cups/*
You should also run rm -rf /var/cache/cups
You should also run rm -rf /var/spool/cups
...
```

Other times, software might jump versions. For example, OpenBSD lets you install multiple versions of PHP, Python, and other scripting languages, but after upgrading, you must decide which is your preferred default.

```
--- +python-2.7.3p1 -------------------
If you want to use this package as your default system python, as root
create symbolic links like so (overwriting any previous default):
 ln -sf /usr/local/bin/python2.7 /usr/local/bin/python
 ln -sf /usr/local/bin/python2.7-2to3 /usr/local/bin/2to3
 ln -sf /usr/local/bin/python2.7-config /usr/local/bin/python-config
 ln -sf /usr/local/bin/pydoc2.7  /usr/local/bin/pydoc
...
```

You may also need to change some system configuration after an upgrade.

```
--- +tk-8.5.12 -------------------
You may wish to add /usr/local/lib/tcl/tk8.5/man to /etc/man.conf
...
```

The package maintainers put these messages into the package for your benefit. Read them, and if you're in doubt, follow their instructions.

If all of your software is installed via packages, the upgrade process should be painless and transparent. I would like to tell you about all the problems and edge cases, except I've never been able to trigger any. Upgrading packages from the official release media just works. Packages built from the ports tree, however, are more complicated to upgrade, as I discuss in "Upgrading Ports" on page 368.

Why Build Your Own OpenBSD?

Building your own upgrade from OpenBSD's source is intended only for advanced users and those interested in developing OpenBSD and advanced users. You must be comfortable reading and compiling source code, debugging problems, and restoring from backup before even trying to build OpenBSD from source. In OpenBSD's earlier days, I found upgrading from source to be the easiest way to move forward, but now upgrading via snapshot is much easier and much less error-prone. If there's any way you can use an official binary release to install what you're looking for, do so.

When you build OpenBSD from source, you are building the distribution sets that you installed via FTP or CD. The files might not be bundled up in distributions, but their contents will be the same. You still must merge your configuration files with `sysmerge`, and you will still need to update your installed software. The only thing building OpenBSD gets you is the latest version of the branch you're following.

I know of three reasons to build your own OpenBSD from source: to get OpenBSD-stable, to get the latest OpenBSD-current, or to highly customize OpenBSD. OpenBSD-stable is available only as source code. No official installation media is available for -stable, so if you want to go from 5.4-release to 5.4-stable, you must build it from source. If you want the absolutely latest OpenBSD code—newer than the latest snapshot—you must build it. And if you want to highly customize OpenBSD, you must build it from source.

Preparations for Building Your Own OpenBSD

Building OpenBSD takes up about 4GB of disk, 2GB in */usr/src* and 2GB in */usr/obj*. OpenBSD creates these directories as separate partitions by default, but if you designed your own disk-partition scheme, make sure you have enough space.

You must build on your target platform. Don't try to cross-compile by, for example, building VAX binaries on an amd64 box. OpenBSD developers use cross-compiling to bring up a new hardware platform, but once the platform is up and running, all development happens on the native hardware. Code produced by cross-compiling might not be identical to code produced by the compiler running on the native platform.

Preparing the Base Operating System

Your first step is to prepare the base operating system. You can build OpenBSD on only an OpenBSD installation similar to what you're trying to build, which means you need to install the closest binary set to your target platform on your build machine. If you're trying to build OpenBSD 5.4-stable, start by installing 5.4-release on your build machine. If you're trying to build the latest OpenBSD-current, upgrade your build machine to the newest snapshot first.

Why start from the closest available binary release? For one, that's how the OpenBSD developers do it; the code is meant to be built from a very recent OpenBSD. Although the system changes only slowly, those changes can add up. OpenBSD-stable is built only on the OpenBSD release it's based on, or on a -stable release following that base release.

Also, OpenBSD-current occasionally has "flag days," where a critical system change makes building the source code difficult. These might be compiler or linker upgrades, library or kernel changes, or just about anything else. While the OpenBSD team documents what you need to do to get over these humps, those documents are more like notes for people who know what they're doing than user-friendly instructions.

These flag days are announced on the OpenBSD mailing lists. You can try to follow those notes and build OpenBSD over these barriers, but when you get sick of beating your head against a flag-day change, upgrade to a snapshot to get over the hump. (Many OpenBSD developers also use snapshots to get over flag-day changes; inability to compile OpenBSD through a flag day is not a threat to your manhood or womanhood.)

Getting Source Code

Now get the OpenBSD source code. If you have already installed these files, and you need to upgrade to a more recent version, see "Updating Source Code" on page 385.

For those installing the source code for the first time, the OpenBSD project provides recent code snapshots in four compressed tar files: the userland (*src.tar.gz*), kernel (*sys.tar.gz*), X Windows (*xenocara.tar.gz*), and ports (*ports.tar.gz*). All must be kept synchronized, which means that you cannot use a -current ports tree atop a -stable userland and kernel. Get the file that's closest to what you want to build.

If you're building -stable, you can grab all four files from the release directory of the binary you installed. For example, if you're building OpenBSD 5.4-stable, you'll find *src.tar.gz*, *sys.tar.gz*, *xenocara.tar.gz*, and *ports.tar.gz* on your CD or on an OpenBSD mirror under */pub/OpenBSD/5.4/*.

If you're building -current, you're better off starting with a newer snapshot of the source code. On the OpenBSD mirror sites, under */pub/OpenBSD*, you'll find recent copies of *src.tar.gz* and *sys.tar.gz*. Grab the most recent ones of those. You still need the *xenocara.tar.gz* and *ports.tar.gz* files from the most recent OpenBSD release, as those files are not updated as frequently.

Verify that the directories */usr/src*, */usr/obj*, */usr/xenocara*, */usr/xobj*, and */usr/ports* are empty. Then extract *src.tar.gz* and *sys.tar.gz* under */usr/src*, and Xenocara and *ports* directly under */usr*. Here, I've copied all the tarballs to my home directory:

```
# cd /usr/src
# tar -xzpf $HOME/src.tar.gz
# tar -xzpf $HOME/sys.tar.gz
# cd /usr
# tar -xzpf $HOME/ports.tar.gz
# tar -xzpf $HOME/xenocara.tar.gz
```

This gives you a known good base to start from. If you're looking to build your own OpenBSD, you probably don't want to build a version that's available for installation now; you want to build a version that's not available for installation. This means you want to update the source code to either -current or -stable.

Updating Source Code

OpenBSD uses Concurrent Versions System (CVS) for source code management. CVS is an older, somewhat traditional version control system. While many projects have moved from CVS to newer, shinier, and perhaps more glamorous systems, CVS meets the OpenBSD Project's needs.

OpenBSD has a single central source-code repository: the master CVS server. Only developers and mirror sites have access to this server. When a developer wants to make new OpenBSD code available, he *commits* the changes to this central repository, and other developers can see his changes at this point.

The master CVS server tracks all changes ever made to the OpenBSD source, as well as who made those changes. Mirror sites capture these changes and make them available to the users in a matter of hours. As a user, you can use CVS to update your local source code to the latest version from a mirror site.

Source Code Repositories and Tags

OpenBSD's source code is divided into *repositories*, or collections of code for particular subsystems. You need to download and synchronize only the collections that you want to use.

The OpenBSD CVS repository includes four collections: `src` (userland and kernel source), `ports` (the ports collection), `xenocara` (X Windows), `www` (the website), and the obsolete `X11` and `XF4` X Windows collections. While X11 and XF4 are still kept for historical purposes, you should never need to use that code. The `www` repository is of interest to the website editors and contributors. In order to build a recent OpenBSD, you must update your `src`, `ports`, and `xenocara` collections to the latest versions of your chosen branch.

A *tag* is a label for a particular version of a repository. OpenBSD uses tags to differentiate between -stable, -release, and -current. For example, the source code of every single OpenBSD release ever made includes the file */usr/src/etc/master.passwd*. But the version of *master.passwd* shipped with OpenBSD 2.0 differs wildly from that shipped with OpenBSD 5.3! CVS uses tags to differentiate the various versions of this and every other file. By using tags, you can ask CVS for the version of any file that shipped with any OpenBSD release.

The tag for any -stable version of OpenBSD is the string `OPENBSD` and the version number, separated by underscores. For example, the tag for OpenBSD 5.4 is `OpenBSD_5_4`. Note the two underscores: there is no `OPENBSD_54` or `OPENBSD5_4`. If you use a tag that does not exist, rather than updating your local source code files, you'll delete them. (While you should download the source code for a release, rather than updating to it via CVS, an OpenBSD release appends the string `_BASE` to the tag, as in `OPENBSD-5_4_BASE`.) That said, OpenBSD-current is special in that it has no tag and it includes the latest version of all source code in all repositories.

All of the repositories are designed to be updated synchronously. If, for example, you update the source code repository to -current and leave your ports and Xenocara at their -release or -stable versions, your system will behave unpredictably. As with learning to juggle chainsaws, you're welcome to try it, but don't whine.

CVS Mirrors

You can use anonymous CVS to update your source code to the latest version. Start by choosing a convenient anonymous CVS server from the list at *http://www.OpenBSD.org/anoncvs.html*. There are mirrors all over the world, but choose one on your continent for better performance. (I use *anoncvs13.usa.openbsd.org*[3] in my examples.)

Tracking OpenBSD-current is slightly different than tracking OpenBSD-stable. We'll start with -stable, and then see how -current differs.

Updating to -stable

The first time you update your source code, you must specify the anonymous CVS server on the command line.

```
# cd /usr
# cvs -qd anonymous@server:/cvs get -rOPENBSD_tag -P src
```

CVS is very picky about the order of its command-line arguments, and many flags are position-dependent. Don't change the order of arguments unless you know what you're doing.

Substitute your preferred server and the release tag you want to get into this command. For example, to update my OpenBSD 5.1 source tree to the latest stable version from *anoncvs13.openbsd.org*, I would run the following:

```
# cd /usr
# cvs -qd anonymous@anoncvs13.openbsd.org:/cvs get -rOPENBSD_5_1 -P src
The authenticity of host 'anoncvs13.usa.openbsd.org (192.0.2.217)' can't be
established.
❶ ECDSA key fingerprint is d3:b2:b5:68:87:3b:f6:93:21:fd:28:ea:cc:b6:e1:13.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'anoncvs1.usa.openbsd.org,149.20.54.217' (ECDSA) to
the list of known hosts.
```

Because OpenBSD runs CVS over SSH the first time you run `cvs`, it asks you to verify the CVS mirror's host key. Compare the ❶ key fingerprint given by your client to the fingerprint listed on the CVS mirror list. If they match, you're talking to the actual CVS server. If the fingerprints don't match, something is wrong, which could be a problem at the server, a skipped website update, or an actual security problem at the mirror. If the key fingerprints

3. Again, there is no *anoncvs13.usa.openbsd.org*. Find a server close to you. Stop blindly copying my examples. It's like you think I know what I'm doing or something!

don't match, choose a different CVS mirror. (SSH will cache this key, and future updates won't ask you to verify the key unless it changes.)

There's a pause when CVS starts comparing the source code on your disk to the source code on the server, and just about when you think that something has gone wrong, you'll probably see actual source code updates, like this:

```
U bin/systrace/intercept-translate.c
U lib/libssl/src/crypto/mem.c
U lib/libssl/src/crypto/asn1/a_d2i_fp.c
U lib/libssl/src/crypto/buffer/buffer.c
U sys/conf/newvers.sh
U sys/netinet6/ip6_output.c
U usr.sbin/nsd/query.c
```

This is complete CVS update output for updating OpenBSD 5.1 release to -stable. In all, there were seven changes. When the OpenBSD guys say "minimal changes in -stable," they mean it.

Repeat this for */usr/ports* and */usr/xenocara*.

```
# cd /usr
# cvs -qd anonymous@anoncvs13.openbsd.org:/cvs get -rOPENBSD_5_1 -P ports
# cvs -qd anonymous@anoncvs13.openbsd.org:/cvs get -rOPENBSD_5_1 -P xenocara
```

A source tree records the last server updated from, and on subsequent updates, you can drop the server from the command line. The source tree also knows which repository within the server it's supposed to update from, so you don't need to specify it.

```
# cd /usr/src
# cvs -q up -rOPENBSD_5_1 -Pd
```

Run this same command any time you want the latest -stable source code.

Updating to -current

The process for updating your source code to -current is much the same as updating to -stable, but the command line differs slightly. The first time you update your source tree, you must specify the server name.

```
# cd /usr
# cvs -qd anonymous@server:/cvs get -P src
```

This cvs command gives you the latest -current source code. For example, if I want to update my source code from *anoncvs13.openbsd.org*, I run this:

```
# cd /usr
# cvs -qd anonymous@anoncvs13.openbsd.org:/cvs get -P src
```

You should get the SSH key fingerprint verification message, and see the same type of messages that you would get updating to -stable.

The first time you run CVS to update a specific set of source code, cvs records which server the source code came from. For subsequent updates to the latest -current source code, you can skip the server name and shorten the command, as follows:

```
# cd /usr/src
# cvs -q up -Pd
```

Now you need to build the new source code into program code.

You build -stable and -current in the same way, but -current has more potential issues, so we'll start by building -stable.

Building OpenBSD-stable

Building OpenBSD-stable requires building and installing a new kernel, building and installing a new userland, and building and installing the new Xenocara.

Upgrading the Kernel

Building an upgraded kernel is like building a custom kernel, as discussed in Chapter 19. Basically, the process boils down to this:

```
# cd /usr/src/sys/arch/platform/conf/
# config GENERIC
# cd ../compile/GENERIC
# make clean && make
# make install
```

(If I planned to build OpenBSD regularly on this machine, I would script this.)

NOTE *Chapter 19 is full of warnings about difficulties compiling a custom kernel. These don't apply to building the GENERIC kernel on -stable. Remember that -stable includes only security fixes and obviously correct small changes. ABI, API, configuration, and syntax changes are absolutely forbidden. If this kernel build fails, you probably corrupted your source tree somehow. Remove everything in* /usr/src *and start over.*

After building and installing the new kernel, reboot. You must run the new kernel to build the new userland.

Building the Userland

To build and install everything outside the kernel, remove any old builds and re-create the object directories. Also, make sure that the installed OpenBSD system has all of the necessary directories. Skipping this step will make the build fail and corrupt your source tree.

```
# rm -rf /usr/obj/*
# cd /usr/src
# make obj
# cd /usr/src/etc
# env DESTDIR=/ make distrib-dirs
```

Now you can build and install the userland.

```
# cd /usr/src
# make build
```

Building userland can take several days on an antique system, but should take only an hour or two on more modern hardware. When the process finishes, you should have the new userland installed on the local system.

NOTE *Installing -stable simplifies the userland part of the upgrade. There's no need to merge any new /*etc*/ files with* `sysmerge` *(but there's no harm in running* `sysmerge` *either, just to check that your configuration is indeed valid), or to re-create your device nodes, as they won't change. OpenBSD-stable contains very minor changes from the release on which it's based.*

Building Xenocara

The X Window System might or might not change during -stable. If your CVS update shows no changes in -stable, you don't need to rebuild it, but if any of the files in */usr/xenocara* have changed, you're better off rebuilding X. The build process is completely routine for Xenocara-stable.

```
# cd /usr/xenocara
# rm -rf /usr/xobj/*
# make bootstrap
# make obj
# make build
```

This builds and installs the new X on your system.

Building a Release

This is all very nice if you want to upgrade only one machine. But what if you have several machines to upgrade to your custom OpenBSD build? You don't want to go building -stable on your firewalls or your web server.

If you need to upgrade multiple machines, build your OpenBSD on one machine, and install that build on multiple machines by building your own *release*. This will take less time than you needed to build your upgrade and use under a gigabyte of disk space.

A release is what the OpenBSD Project puts on the mirror sites for you to install. It's a few kernels, tarballs containing the userland files, the index file, and so on. Using these files, you can upgrade your other hosts just as you would perform the official media upgrade discussed earlier this chapter. Building a release is the easiest way to upgrade several OpenBSD hosts to identical versions.

Before you can build a release, you must build both the base OpenBSD system and Xenocara. You could skip Xenocara if you don't want any X on any of your hosts, but a surprising number of third-party programs need the X libraries. It's easier to build X and not install it on selected hosts than to go back and rebuild the whole release because you didn't build Xenocara the first time.

The release process installs OpenBSD in a temporary root directory and then bundles that installation into the release tarballs and related files. Next, it repeats the process with the X software. It assumes that you have the OpenBSD source code in */usr/src* and a completed build in */usr/obj*, as well as the Xenocara source in */usr/xenocara* and a completed build in */usr/xobj*. You can change the build process to get around these requirements, as documented in `release(8)`, but you should accept the defaults.

You'll need to define three directories: one to store your release, one for your temporary OpenBSD root directory, and one for your temporary Xenocara root directory. You can reuse the temporary root directories, but I keep them for reference. I use */home/releasedir* as my release directory, */home/destdir* as my temporary OpenBSD root, and */home/xdestdir* as the Xenocara temporary root.

WARNING *You can use any partition with sufficient free space, except for* /mnt *because the release process uses this partition. Similarly, building a release builds ISO and floppy images using the first* `vnode` *device,* /dev/vnd0. *If you have any disk images mounted using that device (see Chapter 9), the release process will fail. If you must mount a disk image while building a release, use a* `vnode` *device other than* /dev/vnd0.

The release process has three steps: bundling the base system, bundling Xenocara, and indexing the results.

Bundling the Base System

OpenBSD's build system includes all of the glue that you need to build a release. First, do a `make build` of your new OpenBSD so that you're running the same version you want to build a release for. Next, define the temporary OpenBSD root and the release directory in your environment as `$DESTDIR` and `$RELEASEDIR`, respectively.

NOTE *Verify that the temporary OpenBSD root and the release directories are empty before you start. While the release process can overwrite files from an old build, the directories might have obsolete files that you don't want included in the new release.*

```
# echo $DESTDIR
/home/destdir
# echo $RELEASEDIR
/home/releasedir
```

Once you have this ready, building the release involves only a few commands.

```
# cd /usr/src/etc && make release
# cd /usr/src/distrib/sets && sh checkflist
```

Take a look in your release directory. You should see the following items:

- Three kernels (*bsd*, *bsd.mp*, and *bsd.rd*)
- Three floppy boot images if you're building for i386, or one if it's for amd64 (other architectures vary)
- Two ISO images
- Five file sets for the OpenBSD base system

These files are functionally identical to those distributed by the OpenBSD project, except they're based on your custom build.

Once you've finished building releases, be sure to remove the `$RELEASEDIR` and `$DESTDIR` variables from your environment because they can mess up other software builds. You can't successfully build Xenocara with them still set.

Bundling Xenocara

As with bundling the base system, you must first complete a Xenocara build. Confirm that your system has the same version of Xenocara installed that you want included in your release, and then set the `RELEASEDIR` and `DESTDIR` environment variables. `RELEASEDIR` should be identical to that used to bundle the base system, but `DESTDIR` should differ.

NOTE *You could reuse `DESTDIR`, but that will erase everything from your temporary base system installation. Leave those files around until you're sure you have a solid release.*

```
# echo $DESTDIR
/home/xdestdir
# echo $RELEASEDIR
/home/releasedir
```

Now go into the Xenocara source and bundle the release.

```
# cd /usr/xenocara
# make release
```

Xenocara isn't much smaller than the base system. It takes a while to bundle up, so this is a good time to go to lunch. When you get back, you should find five new files in your release directory, all tarballs beginning with *x*.

Before you quit for the day, remove `RELEASEDIR` and `DESTDIR` from your environment.

Indexing the Release

Copy your release to your local FTP or web server, and create an index of the contents. (Only HTTP installs and upgrades need an index file.) The OpenBSD installer and upgrade software will use this index during the installation.

```
# /bin/ls -l > index.txt
```

Make the web or FTP site accessible from the machines you want to upgrade.

Using the Release

Upgrade or install using this release just as you would use an official release. Copy *bsd.rd* to a machine to be upgraded, and then boot into that kernel. When the installer asks where to get the file sets, give the location of your release. Extract and reboot!

Building OpenBSD-current

Building OpenBSD-current is exactly like building -stable, except when it isn't. OpenBSD-current can change wildly, and building it from source is considered an advanced activity. The only time I actually build -current is if I need to test some new functionality or a patch offered by an OpenBSD developer.

The two big problems are the radical changes in -current and merging */etc*.

Following -current

When you follow -current, keep a close eye on OpenBSD's changes by tracking the Following -current web page at *http://www.OpenBSD.org/faq/current.html*. This is where the OpenBSD developers list all of the changes likely to impact people trying to build -current. Not all changes apply to all systems, but any change listed that applies to your system requires special handling.

The entries are chronological, and include everything since the last release. Just concern yourself with entries dated on or after the date of the source code used to build your snapshot. For example, if you built a snapshot dated January 30, and you want to build -current on the following February 5, check the web page for any entries between those two dates, inclusive. Earlier changes are already incorporated in the installed snapshot.

Some changes will require your intervention before you even try to build the system. For example, if you have new unprivileged usernames, they will need to be in place before a `make build` can succeed—after all, a program owned by user `_fdisk` can't be installed unless that user is in place.

If you don't understand an entry on this page, do not upgrade!

Merging /etc

When you upgrade to a new -stable, you can be sure that the files in */etc/* haven't changed. When you track -current, those critical system files might well change. Any critical changes are usually noted in the Following -current website, but it's best to use `sysmerge(8)` to merge in all */etc* changes. You can give `sysmerge` the path to the system source instead of the *etc.tgz* file set.

```
# sysmerge -s /usr/src
```

See the section about `sysmerge` earlier in this chapter for detailed information.

Upgrading Ports

If you use OpenBSD-provided packages, upgrading your system is as easy as running `pkg_add -ui`. If you built your third-party packages from source using the ports collection, however, there's no easy way to upgrade. You must rebuild those packages. There is no automated way to do this, but the `make update` command in a port can rebuild a specific port.

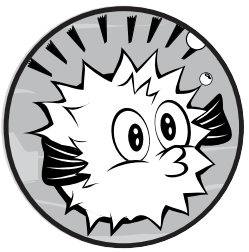
Presumably, you built your own packages because the OpenBSD-provided packages lacked some option or flavor you needed. In that case, you probably only needed to build one or two packages from source. All of the software required by that package could be installed from official OpenBSD sources. You should upgrade everything possible via packages and rebuild only what is strictly necessary.

Now that you can upgrade OpenBSD any way you want, let's look at OpenBSD's packet filter.

21

PACKET FILTERING

The name's Pond, James Pond.
My x86 loaded,
licensed to filter.



Packet filtering and traffic manipulation are among the most basic tools in network security. OpenBSD includes a very powerful in-kernel packet filter, `pf(4)`, or PF. This tool not only performs standard filtering, but it can also inspect, reassemble, redirect, and otherwise abuse packets in several ways; translate addresses in several different directions simultaneously; authenticate users; and manage bandwidth. Along with PF, OpenBSD includes programs that let you turn your system into a load balancer, transparent proxy, or any number of other network devices.

PF is one of the high points of OpenBSD and deserves its own book. That book is *The Book of PF, 2nd edition*, by Peter Hansteen (No Starch Press, 2010), which goes into detail on many different PF use cases. This chapter covers the basics of PF so that you can protect a small network or an individual server. If you want to protect a web farm and transparently relay traffic to only the active servers with sufficient free capacity to handle the load, get Peter's book.

That said, not even Peter's book covers PF in its entirety. OpenBSD lets you fold, spindle, and mutilate TCP/IP far beyond anything any reasonable person could ever expect to support in the real world. For complete details on PF, read the `pf(4)`, `pfctl(8)`, and `pf.conf(5)` man pages, and the OpenBSD PF FAQ at *http://www.OpenBSD.org/faq/pf/*.

NOTE *PF is still undergoing active development. While the configuration syntax doesn't change as often as it used to, check `pf.conf(5)` for the latest information on your version of OpenBSD.*

Firewalls

The word *firewall* has been tortured beyond recognition over the past 20 or so years, until it has ceased to mean much of anything in particular. In general, a firewall sits between a private and public network, and controls the traffic between the two.

You can buy a firewall for your cable modem for under $100, and you can purchase an enterprise firewall cluster for $1 million. What's the difference? They're all firewalls, much as rats and cats and elephants are all mammals, but some are welcome in your home and most are not.[1] Which you permit, of course, is your personal preference. And firewalls are much the same.

Some firewalls filter application-level traffic. Some only filter based on protocol or ports. Some firewalls inspect protocol flags and ensure traffic sanity. Others just pass packets. And some firewalls just translate network addresses and claim that provides security. Worse, the price tag bears no relationship to the feature set.

At their most basic, all firewalls filter packets and can perform network address translation (NAT). OpenBSD can perform those tasks as well or better than most commercial firewalls. If you want application proxies, however, they don't come with the core OpenBSD system (with the exception of FTP and TFTP proxies, which are necessary for those protocols to function with NAT). Several popular application proxies run quite well on OpenBSD, but they are not part of OpenBSD. For example, I've used Squid (*/usr/ports/www/squid*) and several related packages to build a web proxy and filter on OpenBSD that is comparable to anything the big companies offer, and an assortment of other proxies to manage just about everything else. If you are interested in firewalls, I highly recommend that you assemble your own highly featured firewall from available components at least once, for the sake of education if nothing else.

A firewall is what you make it. You can send all your traffic through a simple OpenBSD packet filter and honestly declare that you have a firewall, or you can set up application proxies, authentication, and so on, and still

1. Sorry, cats and elephants, find your own place to live.